A project report on

CASHEW: A DECENTRALIZED PROTOCOL FOR WEB3-NATIVE SUBSCRIPTION LIFECYCLES

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Computer Science and Engineering

by

VARDHMAN BENGANI (23BCE5082)
VEDANT SHARMA (23BCE5094)
ISHITA SINGH (23BCE1955)



VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

October, 2026

CASHEW: A DECENTRALIZED PROTOCOL FOR WEB3-NATIVE SUBSCRIPTION LIFECYCLES

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Computer Science and Engineering

by

VARDHMAN BENGANI (23BCE5082)
VEDANT SHARMA (23BCE5094)
ISHITA SINGH (23BCE1955)



VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

October, 2026

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

DECLARATION

I hereby declare that the thesis entitled “Cashew: A Decentralized Protocol for Web3-Native Subscription Lifecycles” submitted by Vedant Sharma (23BCE5094), for the award of the degree of Bachelor of Technology in Computer Science and Engineering, Vellore Institute of Technology, Chennai is a record of bonafide work carried out by me under the supervision of Prof. Saktivel R.

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Chennai

Date: **Signature of the Candidate**

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

School of Computer Science and Engineering

CERTIFICATE

This is to certify that the report entitled “Cashew: A Decentralized Protocol for Web3-Native Subscription Lifecycles” is prepared and submitted by Vedant Sharma (23BCE5094) to Vellore Institute of Technology, Chennai, in partial fulfillment of the requirement for the award of the degree of Bachelor of Technology in Computer Science and Engineering is a bonafide record carried out under my guidance. The project fulfills the requirements as per the regulations of this University and in my opinion meets the necessary standards for submission. The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma and the same is certified.

Signature of the Guide:

Name: Prof. Saktivel R

Date:

Signature of the Examiner

Name:

Date:

Signature of the Examiner

Name:

Date:

Approved by the Head of Department,

B.Tech. Computer Science and Engineering

Name: Prasanna J

Date:

(Seal of the School of Computer Science and Engineering)

ABSTRACT

Recurring payment is the one everyday financial primitive that public blockchains still borrow from conventional finance: a smart contract cannot act on a timer, and the usual workaround, a large standing ERC-20 allowance, can be drained in a single transaction. This project presents Cashew, a protocol for subscription lifecycles that needs no trusted operator. A provider publishes a plan with immutable price, period, token and payee. A subscriber enrols with one EIP-712 permit signature, and afterwards any account, acting as an untrusted keeper, may trigger each charge when it falls due. The contract itself enforces the limit, at most the plan price at most once per elapsed period on an anchored schedule, and pays the keeper 0.25 % of each successful charge.

The subscription is demonstrated as paid access to AI Power Grid, a live decentralized inference network on which the team operates its own GPU worker serving qwen3 models. A Cashew Gateway serves the OpenAI-compatible API only to wallets whose on-chain subscription is entitled, meters every request, and commits a Merkle root of usage rows on-chain each epoch.

Cashew was implemented as four Solidity contracts, a Node.js keeper, an Express gateway and a Next.js dashboard, and deployed and verified on Ethereum Sepolia. It is backed by 72 Foundry tests, including fuzz properties and five handler-based invariants, with 100 % line coverage. Measurements show batching lowers the charge cost from 71,595 gas to 27,567 gas per subscriber, under 0.003 % of a 20-dollar cycle at the measured Base Sepolia gas price. In a 500-subscriber, twelve-period simulation, all 5,500 renewals executed for keeper intervals from one minute to six hours. After a three-period keeper outage, all 60 missed charges were recovered. A full lifecycle and live, on-chain-verified metering were demonstrated on the public testnet.

Keywords: recurring payments, smart contracts, keepers, EIP-712, decentralized AI inference, Merkle commitments.

ACKNOWLEDGEMENT

It is my pleasure to express with deep sense of gratitude to Prof. Saktivel R, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, for their constant guidance, continual encouragement, understanding; more than all, they taught me patience in my endeavor. My association with them is not confined to academics only, but it is a great opportunity on my part of work with an intellectual and expert in the field of blockchain and distributed systems.

It is with gratitude that I would like to extend my thanks to the visionary leader Dr. G. Viswanathan our Honorable Chancellor, Mr. Sankar Viswanathan, Dr. Sekar Viswanathan, Dr. G V Selvam Vice Presidents, Dr. Sandhya Pentareddy, Executive Director, Ms. Kadhambari S. Viswanathan, Assistant Vice-President, Dr. V. S. Kanchana Bhaaskaran Vice-Chancellor, Dr. T. Thyagarajan Pro-Vice Chancellor, VIT Chennai and Dr. P. K. Manoharan, Additional Registrar for providing an exceptional working environment and inspiring all of us during the tenure of the course.

Special mention to Dr. Viswanathan V, Dean, Dr. Nithyanandam P, Associate Dean, Dr. Suganya G, Associate Dean, Dr. Sweetlin Hemalatha C, Associate Dean, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai for spending their valuable time and efforts in sharing their knowledge and for helping us in every aspect.

In jubilant state, I express ingeniously my whole-hearted thanks to **Prasanna J**, Head of the Department, B.Tech. **Computer Science and Engineering** and the Project Coordinators for their valuable support and encouragement to take up and complete the thesis.

My sincere thanks to all the faculties and staffs at Vellore Institute of Technology, Chennai who helped me acquire the requisite knowledge. I would like to thank my parents for their support. It is indeed a pleasure to thank my friends who encouraged me to take up and complete this task.

Place: Chennai

Date: **Vedant Sharma**

CONTENTS

ABSTRACT.....i
ACKNOWLEDGEMENT.....ii
LIST OF TABLES.....vi
LIST OF FIGURES.....vii
LIST OF ACRONYMS.....viii

CHAPTER 1 INTRODUCTION.....1
1.1 INTRODUCTION.....1
1.2 BACKGROUND.....1
1.2.1 RECURRING PAYMENTS IN WEB2.....1
1.2.2 WHY A BLOCKCHAIN CANNOT BILL BY ITSELF.....1
1.3 OVERVIEW OF AI POWER GRID.....2
1.4 CHALLENGES.....3
1.5 PROBLEM STATEMENT.....4
1.6 OBJECTIVES.....4
1.7 SCOPE OF THE PROJECT.....4
1.8 CONTRIBUTIONS.....5
1.9 ORGANISATION OF THE REPORT.....5

CHAPTER 2 BACKGROUND AND LITERATURE SURVEY.....7
2.1 INTRODUCTION.....7
2.2 BLOCKCHAINS AND SMART CONTRACTS.....7
2.3 TOKENS AND STABLECOINS.....7
2.4 AUTHORISATION PRIMITIVES.....8
2.4.1 TYPED SIGNATURES AND PERMITS.....8
2.4.2 META-TRANSACTIONS AND ACCOUNT ABSTRACTION.....8
2.4.3 SIGN-IN WITH ETHEREUM.....9
2.5 RECURRING AND BATCHED PAYMENT DESIGNS.....9
2.6 ORACLES AND KEEPERS.....9
2.7 AUTHENTICATED USAGE RECORDS.....10
2.8 COMPARISON OF RELATED WORK.....10
2.9 RESEARCH GAP.....10

CHAPTER 3 SYSTEM REQUIREMENTS AND ANALYSIS.....12
3.1 STAKEHOLDERS.....12
3.2 FUNCTIONAL REQUIREMENTS.....12

3.3 NON-FUNCTIONAL REQUIREMENTS....13
3.3.1 SECURITY....13
3.3.2 COST....13
3.3.3 LIVENESS AND TIMELINESS....13
3.3.4 USABILITY....13
3.3.5 PROTECTION OF THE SHARED AI CREDIT....14
3.4 HARDWARE AND SOFTWARE REQUIREMENTS....14
3.5 FEASIBILITY ANALYSIS....14
3.5.1 TECHNICAL FEASIBILITY....14
3.5.2 ECONOMIC FEASIBILITY....15
3.5.3 OPERATIONAL FEASIBILITY....15
CHAPTER 4 SYSTEM DESIGN....16
4.1 ARCHITECTURAL OVERVIEW....16
4.2 DATA MODEL....17
4.3 SUBSCRIPTION STATE MACHINE....18
4.3.1 ENTITLEMENT....18
4.4 CHARGING ALGORITHM....18
4.4.1 ANCHORED SCHEDULE....19
4.4.2 BATCHING AND DUE ENUMERATION....19
4.4.3 KEEPER INCENTIVE....19
4.5 ONE-SIGNATURE ENROLMENT....20
4.6 KEEPER SERVICE DESIGN....20
4.7 GATEWAY DESIGN....21
4.8 METERING AND USAGE COMMITMENTS....23
4.9 ONE BILLING PERIOD END TO END....24
4.10 SAFETY PROPERTIES....25
4.11 DESIGN DECISIONS....26
CHAPTER 5 IMPLEMENTATION....27
5.1 REPOSITORY STRUCTURE....27
5.2 SMART CONTRACTS....27
5.2.1 MOCKUSDC....27
5.2.2 PLANREGISTRY....27
5.2.3 SUBSCRIPTIONVAULT....28
5.2.4 USAGEROOTREGISTRY....28
5.3 DEPLOYMENT....28

5.4 KEEPER SERVICE....29
5.5 CASHEW GATEWAY....30
5.6 INTEGRATION WITH AI POWER GRID....30
5.7 DASHBOARD....31
5.7.1 OVERVIEW AND PROVIDER PAGES....31
5.7.2 PLAN AND SUBSCRIPTION PAGE....32
5.7.3 CHAT PAGE AND THE 402 PATH....33
5.7.4 ACTIVITY, ATTACK PANEL AND EPOCH VERIFIER....34
5.8 LOCAL DEVELOPMENT STACK....36

CHAPTER 6 TESTING AND RESULTS....37
6.1 EVALUATION METHODOLOGY....37
6.2 CONTRACT TEST SUITE....37
6.3 GAS COST PER OPERATION....38
6.4 BATCH CHARGING....39
6.5 COST MODEL....40
6.6 SCHEDULE SIMULATION....41
6.7 KEEPER OUTAGE AND RECOVERY....42
6.8 LIFECYCLE ON ETHEREUM SEPOLIA....43
6.9 LIVE INFERENCE THROUGH THE GATEWAY....45
6.9.1 GRID SNAPSHOT....45
6.9.2 LATENCY....45
6.9.3 METERING AND ON-CHAIN VERIFICATION....46
6.10 DISCUSSION....48

CHAPTER 7 CONCLUSION AND FUTURE WORK....50
7.1 CONCLUSION....50
7.2 LIMITATIONS....51
7.3 FUTURE WORK....52

REFERENCES....53

APPENDICES....56
A. SUBSCRIPTIONVAULT.SOL (COMPLETE SOURCE)....56
B. PLANREGISTRY.SOL AND USAGEROOTREGISTRY.SOL....60
C. KEEPER CORE (KEEPER/SRC/KEEPER.TS)....63
D. INVARIANT SPECIFICATIONS (EXCERPT)....66
E. DEPLOYMENT RECORD AND REPRODUCTION COMMANDS....68

LIST OF TABLES

1.1 Scope: components built in this project versus reused third-party components.….5
2.1 Comparison of Cashew with related designs.....10
3.1 Stakeholders and their goals.....12
3.2 Functional requirements.....12
3.3 Software environment used to build and run the system.....14
4.1 Modules and responsibilities.....17
4.2 Usage-row leaf encoding.....23
4.3 Protocol invariants.....25
4.4 What a keeper (or any caller) cannot do.....26
4.5 Design decisions and their rationale.....26
5.1 Gateway HTTP endpoints.....30
6.1 Contract test summary (forge test).....37
6.2 Fuzz properties.....38
6.3 Results of the five keeper-attack tests.....38
6.4 Execution gas per operation (Foundry gas report, unit tests).....39
6.5 Receipt gas of chargeBatch by batch size.....39
6.6 Cost of operations (ETH/USD = 3,000 assumed).....40
6.7 Schedule simulation: 500 subscribers, 12 × 30-day periods.....41
6.8 Lifecycle executed on Ethereum Sepolia (subscription #1, demo plan).....43
6.9 Live latency, qwen3:8b, max_tokens 64 (second run, n = 3 per path).....45
E.1 Ethereum Sepolia deployment (chainId 11155111).....68

LIST OF FIGURES

1.1 The two sides that cannot reach each other, and the edge Cashew adds.....3
4.1 System architecture: participants, off-chain services, on-chain contracts and the AI Power Grid.....16
4.2 Subscription state machine.....18
4.3 Keeper loop.....21
4.4 Gateway request pipeline.....22
4.5 Per-epoch usage commitment and verification.....24
4.6 Message sequence for one billing period.....25
5.1 SubscriptionVault source verified on Sepolia Etherscan (exact match).....29
5.2 Overview page on Sepolia: contracts, keeper and gateway health, counters and plans.....31
5.3 Provider page: plan creation, own plans and revenue withdrawal.....32
5.4 Plan page with an active, entitled subscription after ten charges.....33
5.5 Live chat with qwen3:8b on AI Power Grid through the gateway (reasoning shown muted).....34
5.6 HTTP 402 Payment Required for a wallet with no subscription.....34
5.7 Activity feed: subscriptions, keeper charges with drift, failure, cancellation and usage roots.....35
5.8 Attack panel: all five attacks rejected by the contract with decoded errors.....36
6.1 Median execution gas per operation.....39
6.2 Gas per subscriber and total gas per transaction against batch size.....40
6.3 Charge cost as a fraction of the 20 USD cycle against batch size.....41
6.4 Cumulative distribution of charge drift for the four keeper tick intervals.....42
6.5 Status of subscription #1 over time on Sepolia (numbers refer to Table 6.8).....44
6.6 A keeper charge on Sepolia Etherscan: 20 mUSDC moved from the subscriber to the vault.....44
6.7 Time to first token, direct to AIPG and through the Cashew Gateway.....46
6.8 Epoch verifier: a usage row proven against the root committed on Sepolia.....47
6.9 Epoch verifier: the same row with one extra completion token is rejected.....48

LIST OF ACRONYMS

Acronym	Expansion
ABI	Application Binary Interface
AIPG	AI Power Grid
API	Application Programming Interface
CEI	Checks-Effects-Interactions (ordering pattern)
CDF	Cumulative Distribution Function
DApp	Decentralized Application
DeFi	Decentralized Finance
EIP	Ethereum Improvement Proposal
EOA	Externally Owned Account
ERC	Ethereum Request for Comments
ETH	Ether, the native token of Ethereum
EVM	Ethereum Virtual Machine
GPU	Graphics Processing Unit
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
L1 / L2	Layer 1 / Layer 2 blockchain
mUSDC	Mock USD Coin (6-decimal test stablecoin)
REST	Representational State Transfer
RPC	Remote Procedure Call
SIWE	Sign-In With Ethereum (EIP-4361)
SSE	Server-Sent Events
SQL	Structured Query Language
TTFT	Time To First Token
UI	User Interface
USDC	USD Coin (fiat-backed stablecoin)
bps	Basis points (1 bps = 0.01 %)

Chapter 1

Introduction

1.1 INTRODUCTION

A large and growing share of digital services is sold by subscription: software, media, cloud compute and, increasingly, access to artificial-intelligence models. In conventional finance the mechanics of a subscription are invisible to the customer. A card number is placed on file once, and the merchant's payment processor charges it every month until the customer cancels. The trust chain is long but familiar. The customer trusts the merchant not to overcharge, the merchant trusts the processor, and both trust the bank and the card network to reverse mistakes.

Public blockchains such as Ethereum (Buterin, 2014; Wood, 2014) replaced much of that chain for one-off payments. A transfer of a stablecoin settles in seconds, is final, and needs no intermediary. Recurring payment, however, has no native expression on a blockchain. This project, **Cashew**, is a protocol that gives recurring payments a native, trust-minimised form on Ethereum-compatible chains. To show that it works against something real, the project uses it to sell metered access to a live decentralized AI inference network, AI Power Grid.

1.2 BACKGROUND

1.2.1 RECURRING PAYMENTS IN WEB2

A card-on-file subscription has three properties that users rely on without noticing. It is **automatic**: nobody has to remember to pay. It is **bounded in practice**: if the merchant charges the wrong amount, the bank can reverse the charge. And it is **revocable**: the customer can cancel, and in the worst case block the card. All three properties come from trusted intermediaries rather than from the payment instrument itself.

1.2.2 WHY A BLOCKCHAIN CANNOT BILL BY ITSELF

Smart contracts on Ethereum are passive. Code runs only when a transaction calls it, and a transaction must be signed and paid for by some account. There is no cron job inside the Ethereum Virtual Machine, so “charge this wallet every 30 days” cannot

be written as a contract alone. Something outside the chain has to send a transaction at the right time (Antonopoulos & Wood, 2018).

The obvious workaround has its own problem. The ERC-20 token standard (Vogelsteller & Buterin, 2015) lets a holder approve a spender for an amount. A merchant could ask for an allowance large enough to cover a year of billing and then pull payments as they fall due. But that allowance is a single number with no notion of time. Whoever controls the spender address can pull the entire amount at once. A subscription built this way is safe only as long as the merchant's key and code are perfectly honest and perfectly secure, which is exactly the trust a blockchain is meant to remove.

1.3 OVERVIEW OF AI POWER GRID

AI Power Grid (AIPG) is a permissionless network for AI generation (AI Power Grid, 2026). Owners of graphics processors connect “workers” that serve open-weight models for text, image, video and audio generation. Developers reach those models through an OpenAI-compatible HTTP API at `api.aipowergrid.io`, and the grid routes each job to a worker that serves the requested model. During this project the team connected its own GPU worker to the grid, serving the open-weight models `qwen3:8b`, `qwen3:4b` and `qwen3:1.7b`. The live worker list showed it alongside community workers serving models such as `gpt-oss-120b` and `deepseek-v4-flash`.

AIPG is a natural counterpart for Cashew because it has the same two-sided shape that subscriptions are meant to bridge (Figure 1.1). On one side are GPU owners with idle capacity and no billing relationship with the people who might use it. On the other are developers and autonomous agents who hold crypto assets but have no card to put on file. Neither side can transact with the other without an intermediary today. A subscription contract, charged on-chain each period and checked by a gateway in front of the grid's API, is the missing edge.

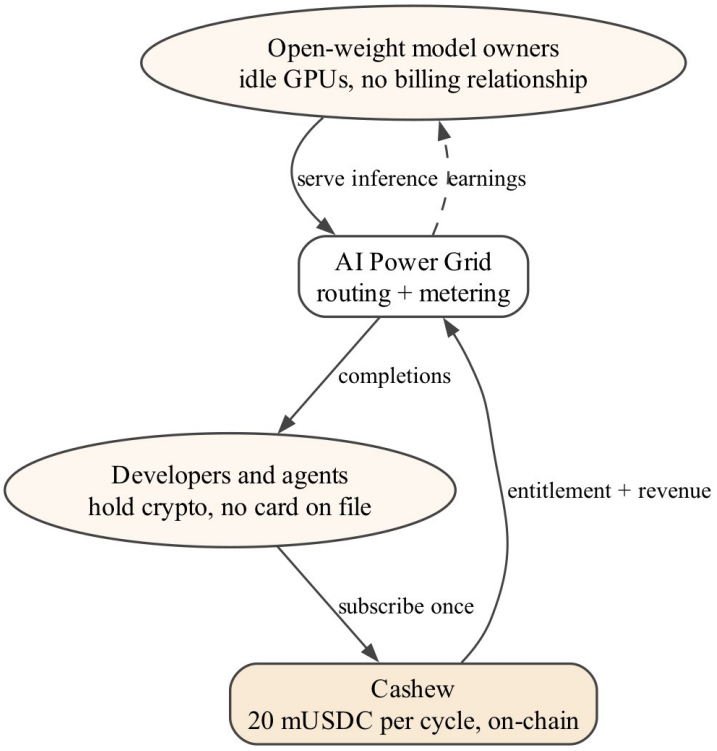


Figure 1.1 The two sides that cannot reach each other, and the edge Cashew adds

1.4 CHALLENGES

Building recurring payment on a public chain raises five challenges, and they shape every design decision in this report.

1. **No clock.** A contract cannot wake itself. Someone must submit each charge, and that someone must have a reason to do so.
2. **Unbounded authority.** A plain allowance gives the spender power over the whole approved amount at once. The spending limit has to live in contract logic, not in the allowance.
3. **Untrusted triggers.** If anyone may submit a charge, then anyone may also try to submit a wrong one: too early, twice, after cancellation, or for the wrong amount.
4. **Usability.** Asking a subscriber for an approval transaction, a subscription transaction and a fresh signature every period would defeat the purpose. Enrolment must be close to a single action.

5. **Accountable service delivery.** When the seller is a network of anonymous GPUs rather than a known company, both sides need evidence of what was actually consumed in each period.

1.5 PROBLEM STATEMENT

Design, implement and evaluate a protocol that allows a subscriber to authorise recurring payments to a provider with a single signature, such that (i) any party may trigger a due payment and is compensated for doing so; (ii) the contract guarantees that no more than the plan price can be collected per elapsed period, regardless of who triggers it; (iii) the subscriber can cancel at any time and retains access for the period already paid; and (iv) access to a real metered service is granted only while the subscription is entitled, with per-period usage committed on-chain in a verifiable form.

1.6 OBJECTIVES

- Specify and implement the protocol as four Solidity contracts: a settlement token with permit, a plan registry with immutable terms, a subscription vault that enforces charging rules, and a registry for usage commitments.
- Prove the protocol's safety properties with unit, fuzz and handler-based invariant tests, including five explicit “keeper cannot” properties.
- Build a permissionless keeper service that discovers due subscriptions and charges them in batches.
- Build a gateway that gates an OpenAI-compatible API behind on-chain entitlement, meters usage, and commits Merkle roots of usage per epoch.
- Provide a dashboard covering the full lifecycle, including an attack panel that demonstrates the five keeper attacks failing.
- Measure cost (gas per operation and per subscriber), timeliness (drift and missed charges) and liveness (recovery after keeper outage), and validate the full lifecycle on a public testnet with live inference.

1.7 SCOPE OF THE PROJECT

The project covers the protocol, its tests, the three off-chain services and their deployment on the Ethereum Sepolia testnet (chain id 11155111). Settlement uses a mock 6-decimal stablecoin with an open faucet, so no real money is involved at any

point. The AI service is the public AIPG API, accessed through one platform-level API key held by the gateway. The team does not run AIPG's coordinator software. Table 1.1 separates what was built in this project from what is reused.

Table 1.1 Scope: components built in this project versus reused third-party components

Component	Origin	Role in Cashew
MockUSDC, PlanRegistry, SubscriptionVault, UsageRootRegistry	Built (this project)	On-chain protocol
Keeper service, Cashew Gateway, dashboard, simulations	Built (this project)	Off-chain services and evaluation
GPU worker serving qwen3:8b / 4b / 1.7b	Operated by the team on AIPG's worker software	Supplies the inference that is sold
AIPG public API and job routing (grid-core)	Third party (AI Power Grid)	Routes inference jobs to workers
OpenZeppelin Contracts v5.1, Foundry, viem, wagmi, RainbowKit	Third-party libraries	Building blocks and tooling

Out of scope are mainnet deployment, a security audit, automated payouts from the provider's revenue to individual GPU workers, and running an independent instance of AIPG. These are discussed as limitations and future work in Chapter 7.

1.8 CONTRIBUTIONS

The project makes four contributions. First, it gives a recurring-payment contract in which the spending limit is enforced per period inside the contract, so an approval cannot be over-drawn even by a malicious caller. Second, it makes the trigger an open market: any address may charge a due subscription and is paid a fee from the charge, which makes the schedule self-sustaining. Third, it combines the subscription with a metered service in which each period's usage is committed as a Merkle root that anyone can check. Fourth, it demonstrates the whole design against a working network on a public testnet, with measured rather than assumed costs and timings.

1.9 ORGANISATION OF THE REPORT

Chapter 2 surveys the background and related work and identifies the gap this project fills. Chapter 3 states the requirements. Chapter 4 presents the system design: architecture, data model, state machine, charging algorithm, gateway and metering. Chapter 5 describes the implementation and deployment. Chapter 6 reports testing and

results. Chapter 7 concludes with limitations and future work. The appendices list the contract source code and the deployment record.

Chapter 2

Background and Literature Survey

2.1 INTRODUCTION

This chapter reviews the building blocks Cashew depends on and the prior work closest to it. The survey is organised by theme: the blockchain substrate and its token standards; authorisation primitives that make one-signature enrolment possible; earlier designs for recurring and batched payments; oracle and keeper networks that supply off-chain triggers; and authenticated data structures used for usage commitments. The chapter ends with a comparison table and the research gap.

2.2 BLOCKCHAINS AND SMART CONTRACTS

Bitcoin introduced a peer-to-peer ledger in which ownership is established by digital signatures and ordering by proof of work (Nakamoto, 2008). Ethereum generalised the ledger into a replicated state machine whose accounts may hold code (Buterin, 2014). The Ethereum yellow paper specifies the Ethereum Virtual Machine, in which every operation has a gas cost paid by the transaction sender (Wood, 2014). Two consequences of this model matter for subscriptions. First, a contract executes only inside a transaction that someone else submitted, so contracts are passive. Second, every state change has a measurable cost, so the cost of a charge, and how batching amortises it, can be stated precisely.

Smart contracts have well-documented failure modes. Atzei, Bartoletti and Cimoli (2017) catalogue attacks on Ethereum contracts, including reentrancy, unchecked external calls and transaction-ordering dependence. Daian et al. (2020) show that pending transactions are public and can be front-run by bots that observe the mempool. Both results influence Cashew's design: all value-moving functions follow the checks-effects-interactions ordering and use a reentrancy guard, and the permit signature is handled so that a front-run copy of it cannot block enrolment.

2.3 TOKENS AND STABLECOINS

The ERC-20 standard defines the fungible-token interface used by almost every payment token on Ethereum, including the approve and transferFrom pair that lets a

third party move tokens within an allowance (Vogelsteller & Buterin, 2015). A subscription priced at a fixed fiat amount is meaningful only in a unit with stable purchasing power. Mita et al. (2019) survey the price-stabilisation mechanisms of stablecoins, and Pernice et al. (2019) analyse their design approaches and open questions. Cashew therefore settles in a USD-denominated stablecoin. The testnet deployment uses a 6-decimal mock with the same interface as USDC.

Lu et al. (2021) collect design patterns for blockchain-based payment applications, organised around the life cycle of a token. Cashew's plan registry corresponds to their idea of a template contract instantiated per use, and its pull-payment withdrawal follows the well-known pattern of crediting balances and letting each party withdraw its own. In the broader decentralized-finance landscape surveyed by Gogol et al. (2024), recurring payment is notably absent as a primitive.

2.4 AUTHORISATION PRIMITIVES

2.4.1 TYPED SIGNATURES AND PERMITS

EIP-712 defines how to hash and sign typed, structured data so that a wallet can show the user exactly what they are approving instead of an opaque hash (Bloemen, Logvinov, & Evans, 2017). ERC-2612 builds on it with a permit function: the token holder signs an approval off-chain and anyone may submit it, so an approval no longer needs its own transaction (Lundfall, 2020). Cashew uses permit for enrolment. The subscriber signs one typed message, and the subscribe transaction submits it.

2.4.2 META-TRANSACTIONS AND ACCOUNT ABSTRACTION

A subscriber who holds only stablecoins still needs native currency for gas. Seres (2020) formalises meta-transactions, in which a relayer pays gas on the user's behalf, and argues that with sufficient market liquidity they do not weaken platform security. Onica and Amariei (2022) move the relaying logic into an Intel SGX enclave to reduce trust in the relayer. ERC-4337 standardises account abstraction through an alternative mempool and paymasters that can sponsor gas (Buterin et al., 2021), and Wang and Chen (2023) give a first systematic analysis of its security properties. These approaches are complementary to Cashew. The subscriber in Cashew signs once and sends one transaction, and every later charge is paid for by the keeper, not the subscriber. Sponsoring that first transaction through a paymaster is left as future work.

2.4.3 SIGN-IN WITH ETHEREUM

EIP-4361 specifies a human-readable message format by which a wallet proves control of an address to an off-chain service, including a domain, a nonce and an expiry (Chang et al., 2021). Cashew's gateway uses it to bind API sessions to a wallet without passwords.

2.5 RECURRING AND BATCHED PAYMENT DESIGNS

ERC-948 opened the discussion of a subscription standard for Ethereum (Owocki, 2018). The proposals that followed authorise a standing pool of funds from which the merchant's contract may draw on a schedule. They leave the enforcement of timing and amount to the merchant's contract and the triggering to a merchant-run service. Grybniak et al. (2022) propose recurring payments on EVM platforms as a token contract backward-compatible with ERC-20 and ERC-777, so that the token itself understands recurring transfers. CHAINGE (Buckley et al., 2021) addresses a different subscription problem, keeping card details up to date across services, using a permissioned Hyperledger Sawtooth ledger.

Two lines of work tackle the cost and expressiveness of repeated payments. BatPay bundles many ERC-20 micropayments into one transaction and moves most computation off-chain with a challenge game, cutting per-payment gas (Mayer et al., 2020). Meng and Feng (2025) propose composing payments as stream pipelines of template contracts, trading higher gas for much less custom code. Abadi, Murdoch and Zacharias (2023) define recurring contingent service payment, in which each payment is released only against a verifiable proof that a service, such as proof of retrievability, was delivered. Their work shows that recurring payment and verifiable service delivery belong together.

2.6 ORACLES AND KEEPERS

Because contracts cannot observe time or external data on their own, the ecosystem has developed oracle and automation networks. Chainlink 2.0 describes decentralized oracle networks that deliver data and execute contract functions on behalf of users (Breidenbach et al., 2021). Kaleem and Shi (2021) measure how Chainlink oracles are actually used on Ethereum and find usage dominated by price feeds from a small set of providers. In both cases the caller is a permissioned, registered node set

that is paid by subscription to the network, rather than an open market of anonymous callers paid per action by the contract being called.

2.7 AUTHENTICATED USAGE RECORDS

A Merkle tree commits to a set of records with a single hash such that any record can later be proven to belong to the set with a logarithmic-size proof (Merkle, 1988). Committing one root per period is far cheaper than writing each record on-chain and keeps the records themselves private until a proof is requested. Cashew's gateway uses OpenZeppelin's StandardMerkleTree encoding (OpenZeppelin, 2024), whose double-hashed leaves prevent a leaf from being confused with an internal node.

2.8 COMPARISON OF RELATED WORK

Table 2.1 Comparison of Cashew with related designs

Work	Who triggers renewals	Per-period cap in contract	Caller paid by contract	Usage evidence
ERC-948 proposals (Owocki, 2018)	Merchant service	No (standing allowance)	No	No
Grybniak et al. (2022)	Token logic + caller	Token-level schedule	No	No
CHAINGE (Buckley et al., 2021)	Permissioned ledger	Not applicable (card rails)	No	No
BatPay (Mayer et al., 2020)	Payer / operator	No	No	Off-chain challenge game
Chainlink automation (Breidenbach et al., 2021)	Registered node set	Not applicable	Paid by network subscription	No
Recurring contingent payment (Abadi et al., 2023)	Parties of the exchange	Per proof	No	Proof of service
Cashew (this project)	Anyone	Yes: ≤ price, ≤ once per period	Yes: 0.25 % of price	Merkle root per epoch

2.9 RESEARCH GAP

Each ingredient Cashew needs has been studied separately: typed one-signature approvals, batched payments, automation networks, and recurring payments contingent on service. What prior work does not deliver together is (1) an **open trigger**, so that renewals do not depend on any single server; (2) a **built-in incentive** paid by the contract to whoever triggers, so that strangers actually show up; and (3) a **hard spending limit enforced per period inside the contract**, so that the subscriber can never be over-charged by any caller. Cashew combines the three. It then attaches them

to (4) a **verifiable metering layer** over a decentralized compute market, where the seller is an anonymous network of GPUs rather than a trusted merchant.

- The novelty is the reframing of a scheduling problem as an incentive problem: instead of asking how a contract can schedule itself, ask how to make it worthwhile for anyone to call it on time, and make the contract safe against whatever that caller attempts.
- The demonstration on a live inference network makes the price, the cost of charging and the usage commitments measurable rather than assumed.

Chapter 3

System Requirements and Analysis

3.1 STAKEHOLDERS

Five roles interact with the system. Three of them are human or organisational and two are automated services. The analysis below derives requirements from what each role must be able to do and, equally important, from what each role must be prevented from doing.

Table 3.1 Stakeholders and their goals

Role	Goal	Must be prevented from
Provider (creator)	Publish a plan, receive revenue, close the plan to new subscribers	Changing price, period, token or payee after subscribers enrol
Subscriber	Enrol once, use the service while paid, cancel any time	Receiving service after grace without paying
Keeper (anyone)	Trigger due charges and earn a reward	Charging early, twice, after cancel, more than the price, or redirecting funds
Gateway operator	Sell metered access to AIPG through one platform key	Moving funds; serving wallets that are not entitled
GPU worker (AIPG)	Serve inference jobs routed by the grid	Not applicable (third party)

3.2 FUNCTIONAL REQUIREMENTS

Table 3.2 Functional requirements

ID	Requirement
FR1	A provider can create a plan with token, price, period, grace period, payee and metadata. Grace must be strictly less than period. Terms cannot be modified afterwards.
FR2	A provider can open or close a plan to new subscriptions without affecting existing ones.
FR3	A subscriber can enrol with one EIP-712 permit signature and one transaction. The first period is paid immediately.
FR4	Any account can charge a subscription whose charge time has arrived. A successful charge pulls exactly the plan price and advances the schedule by exactly one period.
FR5	A charge that cannot be collected moves the subscription to PastDue during grace and to Lapsed after grace, without reverting.
FR6	Any account can charge many subscriptions in one transaction. Bad ids are skipped rather than reverting the batch.
FR7	A subscriber can cancel at any time and remains entitled until the paid-through time.
FR8	Payees and keepers withdraw only their own credited balance.
FR9	Keepers can enumerate due subscriptions with bounded-cost paging.

ID	Requirement
FR10	The gateway authenticates wallets with Sign-In With Ethereum or an API key bound to a wallet, and serves the OpenAI-compatible chat API only to entitled wallets. Otherwise it returns HTTP 402 with the on-chain status.
FR11	The gateway records every request as a usage row and commits one Merkle root per epoch on-chain. Anyone can obtain a proof for a row and verify it against the chain.
FR12	A dashboard exposes all of the above, including an attack panel demonstrating that the five keeper attacks fail.

3.3 NON-FUNCTIONAL REQUIREMENTS

3.3.1 SECURITY

- No privileged role can move subscriber funds. There is no admin, upgrade path or pause switch in the payment path.
- All value-moving functions are protected against reentrancy and use SafeERC20.
- Five safety invariants (Section 4.10) must hold under randomised sequences of actions.

3.3.2 COST

- The gas cost of keeping a subscription current must be a negligible fraction of the subscription price, and must fall with batching.

3.3.3 LIVENESS AND TIMELINESS

- If keepers stop, charges are delayed but not lost, and a returning keeper catches up.
- A keeper polling every T seconds should execute each charge within roughly T seconds of its due time.

3.3.4 USABILITY

- One signature to enrol. No action per period from the subscriber.
- The gateway must be a drop-in for OpenAI-compatible clients, including streaming responses.

3.3.5 PROTECTION OF THE SHARED AI CREDIT

- Live inference uses a shared, limited credit balance. The gateway must cap max_tokens, rate-limit each wallet, and enforce a persisted global request budget.

3.4 HARDWARE AND SOFTWARE REQUIREMENTS

Table 3.3 Software environment used to build and run the system

Layer	Technology (version)
Smart contracts	Solidity 0.8.24 (optimizer 200 runs, EVM cancun), OpenZeppelin Contracts 5.1.0
Contract toolchain	Foundry 1.8.5: forge (build, test, coverage, scripts), anvil (local chain), cast
Off-chain runtime	Node.js 22 (code targets ≥ 20), TypeScript 5.6, pnpm 9 workspaces, viem 2.21
Gateway	Express 4.21, better-sqlite3 11.5, @openzeppelin/merkle-tree 1.0.7, jsonwebtoken 9
Dashboard	Next.js 14.2 (App Router), React 18.3, wagmi 2.12, RainbowKit 2.2, Tailwind CSS 3.4
Analysis	Python 3.14 with matplotlib 3.10; Graphviz for diagrams
Networks	anvil (chain 31337) for development and simulation; Ethereum Sepolia (11155111) for deployment
External service	AI Power Grid public API (OpenAI-compatible) with one platform API key

Development and all experiments ran on a single Apple-silicon laptop. The GPU worker connected to AIPG serves the qwen3 family of models through Ollama on the team's own graphics hardware. No special hardware is needed to run Cashew itself: the keeper and gateway are lightweight Node.js processes, and a public RPC endpoint is sufficient for testnet operation.

3.5 FEASIBILITY ANALYSIS

3.5.1 TECHNICAL FEASIBILITY

Every primitive the design relies on (ERC-20 transfers, ERC-2612 permits, EIP-712 signatures, Merkle proofs) is standardised and has audited reference implementations in OpenZeppelin Contracts. The OpenAI-compatible AIPG API means the gateway can stream responses using ordinary server-sent events.

3.5.2 ECONOMIC FEASIBILITY

The cost of a charge is borne by the keeper and reimbursed by a fee of 0.25 % of the price, which is 0.05 USD on a 20 USD plan. Chapter 6 shows the measured gas cost of a batched charge is far below that, so running a keeper is profitable without subsidy.

3.5.3 OPERATIONAL FEASIBILITY

All services are stateless or keep only local state (the gateway's SQLite file). The protocol has no operator: if the team's keeper stops, any other party can take over, and the outage experiment in Chapter 6 shows no charge is lost.

Chapter 4

System Design

4.1 ARCHITECTURAL OVERVIEW

Cashew is organised into two planes (Figure 4.1). The **subscription plane** is on-chain. Four contracts hold every unit of value and enforce every payment rule. The **service plane** is off-chain. A keeper supplies the clock, a gateway sells metered access to AI Power Grid, and a dashboard presents the state to people. Two kinds of chain write bracket each period: the keeper's charge at the start and the gateway's usage root at the end. Everything in between is ordinary API traffic. Off-chain components hold no custody. The keeper can only ask the vault to do what the vault already permits, and the gateway can only commit roots, never move funds.

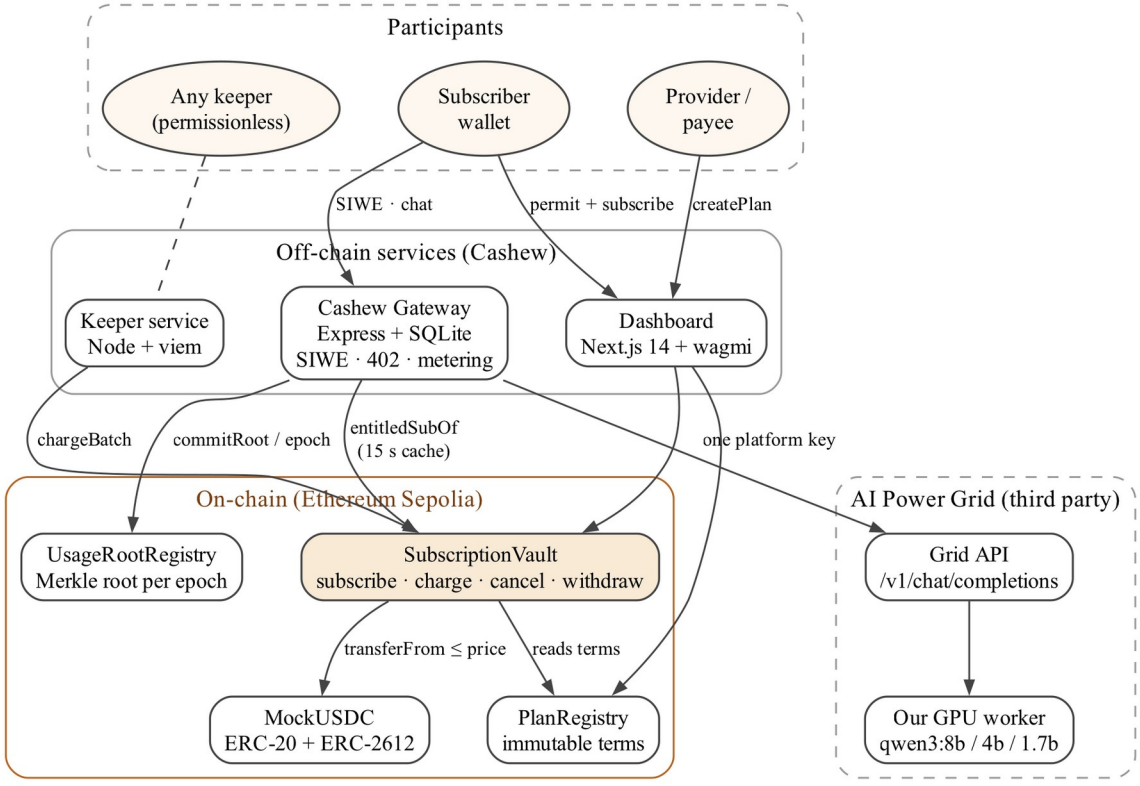


Figure 4.1 System architecture: participants, off-chain services, on-chain contracts and the AI Power Grid

Table 4.1 Modules and responsibilities

#	Module	Responsibility	Implemented in
1	Plan registry	Publish plans with immutable economic terms; open or close to new subscribers	PlanRegistry.sol
2	Subscription vault	Enrol, charge on an anchored schedule, cancel, account balances, entitlement views	SubscriptionVault.sol
3	Open keeper market	Discover due subscriptions, charge in batches, earn the fee	SubscriptionVault.chargeBatch + keeper/
4	One-signature consent	EIP-712 / ERC-2612 permit inside subscribe; front-run tolerant	subscribeWithPermit + MockUSDC
5	Gateway	SIWE auth, API keys, entitlement gate (402), policy, AIPG proxy	gateway/
6	Metering and commitments	Usage rows, per-epoch Merkle roots, proofs, on-chain verification	gateway/ + UsageRootRegistry.sol
7	Dashboard	Provider, plan, chat, activity, attack panel, epoch verifier	dashboard/

4.2 DATA MODEL

A plan and a subscription are the only two records the protocol needs. A plan's economic fields are written once at creation and have no setter. A subscription binds a wallet to a plan and carries the schedule and status. Both structures are packed to minimise storage slots: a subscription occupies two 32-byte slots.

Listing 4.1 Plan and subscription records

```
struct Plan {
    address provider;     // msg.sender at creation
    address payee;        // receives revenue; immutable
    address token;        // ERC-20 settlement token; immutable
    uint96  price;        // per-period price in token units; immutable
    uint32  period;       // seconds; immutable; > 0
    uint32  gracePeriod;  // seconds; immutable; must be < period
    bool    active;       // provider may close to NEW subscriptions only
    string  metadataURI;  // name, description, features
}
enum Status { None, Active, PastDue, Lapsed, Cancelled }
struct Subscription {
    address subscriber;
    uint64  planId;
    uint64  startedAt;
    uint64  nextChargeAt;  // anchored: advances by exactly one period per charge
    uint64  pastDueSince;  // 0 unless PastDue
    uint32  chargeCount;   // includes the payment taken at subscribe time
    Status  status;
}
```

Balances owed to payees and keepers are kept in a mapping `claimable[account][token]`. Funds pulled from subscribers stay in the vault until the entitled account withdraws them. This pull-payment pattern keeps every transfer out of the vault limited to the caller's own balance.

4.3 SUBSCRIPTION STATE MACHINE

Figure 4.2 shows the lifecycle. A subscription starts Active with its first period already paid. A due charge that succeeds keeps it Active and moves `nextChargeAt` forward by exactly one period. A charge that fails for lack of balance or allowance moves it to PastDue if the grace window is still open, recording when the unpaid period began. A later successful charge returns it to Active. If a charge still cannot be collected once grace has ended, the subscription becomes Lapsed. Cancelled and Lapsed are terminal states. A subscriber may then start a fresh subscription to the same plan.

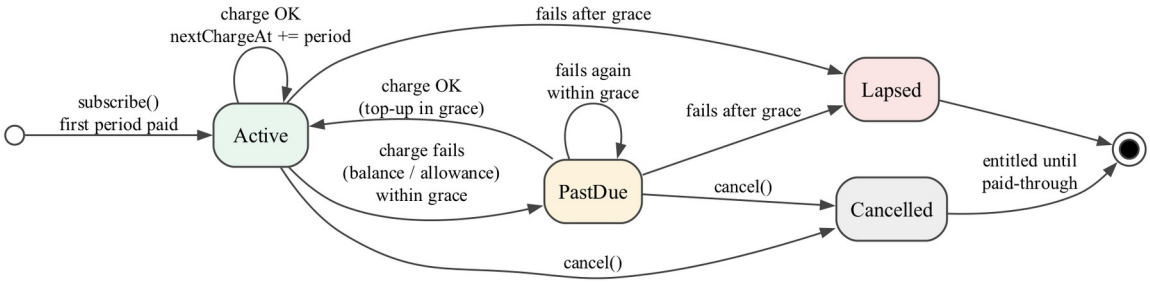


Figure 4.2 Subscription state machine

4.3.1 ENTITLEMENT

Whether a wallet should receive service is computed from the state on every read, never stored:

$$\text{isEntitled} = (\text{status} \in \{\text{Active}, \text{PastDue}\} \wedge \text{now} < \text{nextChargeAt} + \text{grace}) \vee (\text{status} = \text{Cancelled} \wedge \text{now} < \text{nextChargeAt})$$

Three consequences follow. A late keeper does not cut access, because Active subscriptions remain entitled during grace even if nobody has charged them yet. A failed payment cuts access once grace ends, even before any keeper records the lapse. And a cancelled subscription keeps exactly the time it has paid for, with no grace.

4.4 CHARGING ALGORITHM

The charge function is open to every address. Its safety does not come from restricting the caller. It comes from the vault deciding both the amount and the timing itself. Listing 4.2 gives the algorithm.

Listing 4.2 Charging algorithm

```
charge(subId):
    s ← subs[subId]
```

```
    require s.status ∈ {Active, PastDue}            else revert
NotChargeable(s.status)
    require now ≥ s.nextChargeAt                    else revert NotDue()
    t ← terms[s.planId]                             // immutable, cached from
PlanRegistry
    if balance(s.subscriber) < t.price  → reason ← 1
    elif allowance(s.subscriber, vault) < t.price → reason ← 2
    if reason ≠ 0:
        if now ≥ s.nextChargeAt + t.grace: s.status ← Lapsed; emit SubscriptionLapsed;
return
        if s.status ≠ PastDue: s.status ← PastDue; s.pastDueSince ← s.nextChargeAt
        emit ChargeFailed(subId, reason); return       // no revert, no fee
    s.nextChargeAt ← s.nextChargeAt + t.period       // effects first (CEI)
    s.chargeCount ← s.chargeCount + 1; s.status ← Active; s.pastDueSince ← 0
    fee ← t.price × keeperFeeBps / 10 000
    transferFrom(s.subscriber → vault, t.price)
    claimable[t.payee] += t.price − fee; claimable[msg.sender] += fee
    emit Charged(subId, planId, t.price, fee, msg.sender, s.nextChargeAt)
```

4.4.1 ANCHORED SCHEDULE

The schedule advances from the previous due time, not from the time of the call. If the subscription started at time t_0 with period P, then after k keeper charges:

$$\text{nextChargeAt} = t_0 + (k + 1) \cdot P$$

A keeper that is an hour late therefore does not move the billing date by an hour. A keeper that returns after missing several periods settles them one call at a time, and each call collects exactly one period. Because grace is shorter than the period, a subscription that cannot pay lapses at the first attempt after grace instead of accumulating debt. The total ever collected from a subscriber is bounded by

$$\text{collected} \le \text{price} \cdot (1 + \lfloor (\text{now} - t_0) / P \rfloor)$$

which is invariant I1 in Section 4.10.

4.4.2 BATCHING AND DUE ENUMERATION

`chargeBatch(ids)` applies the same per-subscription logic to up to 200 ids in one transaction. Ids that are not chargeable or not yet due are counted as skipped rather than reverting, so one bad id cannot block the rest. Keepers find work through `dueSubscriptions(cursor, limit)`, a view that scans a bounded window of ids and returns those that are due, with a cursor for the next page. This bounds the cost of each call regardless of how many subscriptions exist.

4.4.3 KEEPER INCENTIVE

The keeper's reward is a fixed fraction of the price, `keeperFeeBps = 25` (0.25 %), credited to `msg.sender` only on a successful charge. A failed or skipped charge earns nothing, so a keeper is never paid for spam. On a 20 USD plan the reward is 0.05

USD per charge. Whoever does the work earns it, with no registration, which is what makes the keeper an open market.

4.5 ONE-SIGNATURE ENROLMENT

`subscribeWithPermit(planId, value, deadline, v, r, s)` first calls the token's ERC-2612 `permit` inside a try/catch block, then performs the normal subscription. The subscriber signs a permit for twelve periods (`12 × price`) so that the vault can collect a year of renewals without another signature, but the vault still never pulls more than one price per elapsed period. The try/catch matters because a signed permit is visible in the mempool. If an observer submits the permit first (front-running, see Daian et al., 2020), the subscriber's own transaction would otherwise revert. With the try/catch, enrolment succeeds as long as the resulting allowance is sufficient.

4.6 KEEPER SERVICE DESIGN

The keeper (Figure 4.3) is a stateless loop. At each tick it pages through `dueSubscriptions`, splits the due ids into chunks of `BATCH_SIZE`, simulates and sends `chargeBatch` for each chunk, and parses the resulting events into one log line per subscription. Each line records the scheduled time, the block time of execution, the drift between them, the outcome and the gas used. Within a tick the keeper re-scans until nothing new is due, so a keeper returning from an outage catches up every missed period in one tick. Subscriptions that failed in the current tick are not retried until the next tick.

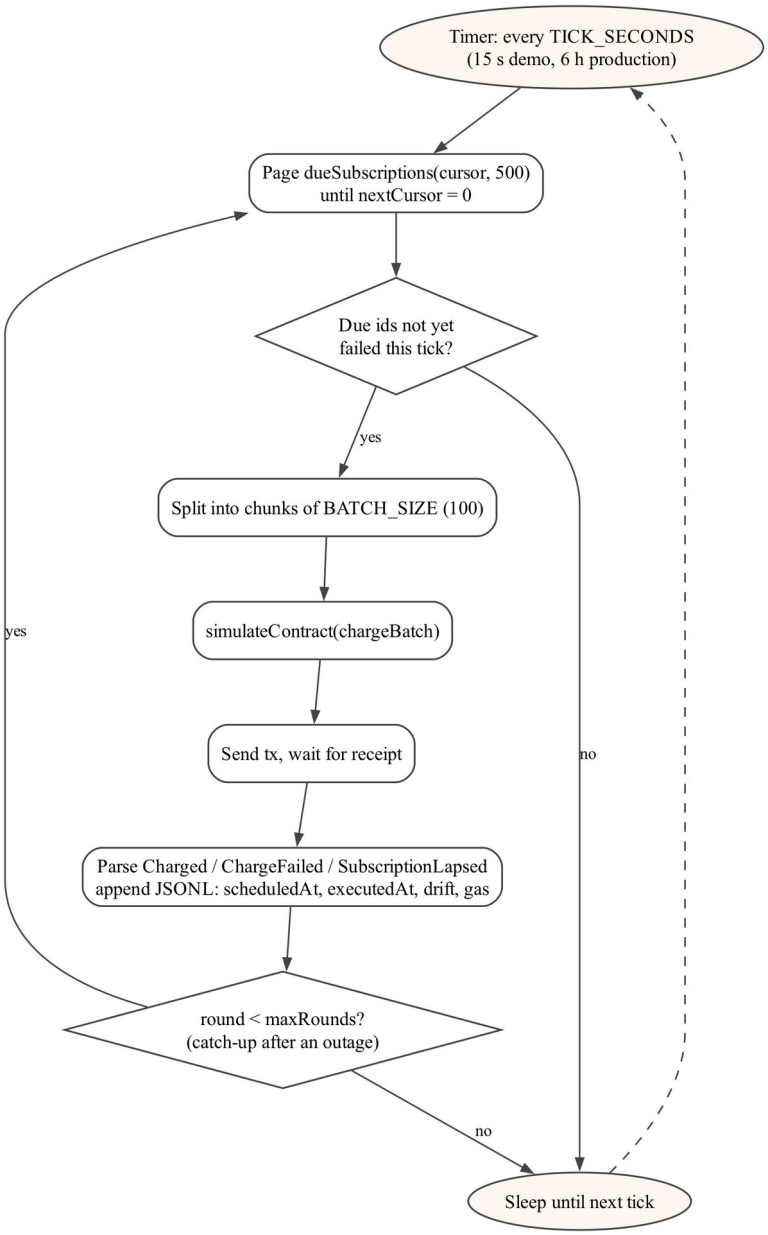


Figure 4.3 Keeper loop

4.7 GATEWAY DESIGN

The gateway (Figure 4.4) is an OpenAI-compatible reverse proxy in front of AIPG. A wallet authenticates once with Sign-In With Ethereum (Chang et al., 2021) and receives a one-hour JSON Web Token, or mints a long-lived `csk_` API key whose SHA-256 hash is stored. API keys cannot mint further keys. For each request the gateway resolves the wallet and checks entitlement on-chain through `entitledSubOf`, cached in memory for 15 seconds. If the wallet is not entitled, it returns **HTTP 402**

Payment Required with the on-chain status, the subscription id and its `nextChargeAt`, so a client can explain why access stopped.

Entitled requests then pass a policy layer: a model allowlist, a cap of 256 on `max_tokens`, a limit of 10 requests per minute per wallet, and a global budget of live requests persisted in SQLite. The budget unit is reserved before the upstream call, so neither restarts nor concurrent requests can overspend the shared credit. The request is forwarded with the platform key. Streamed responses are passed through byte for byte while the gateway accumulates the generated text and the usage block. Upstream calls and a local mock share one code path: the mock returns a synthetic HTTP response with the same shape, so tests exercise the real streaming code.

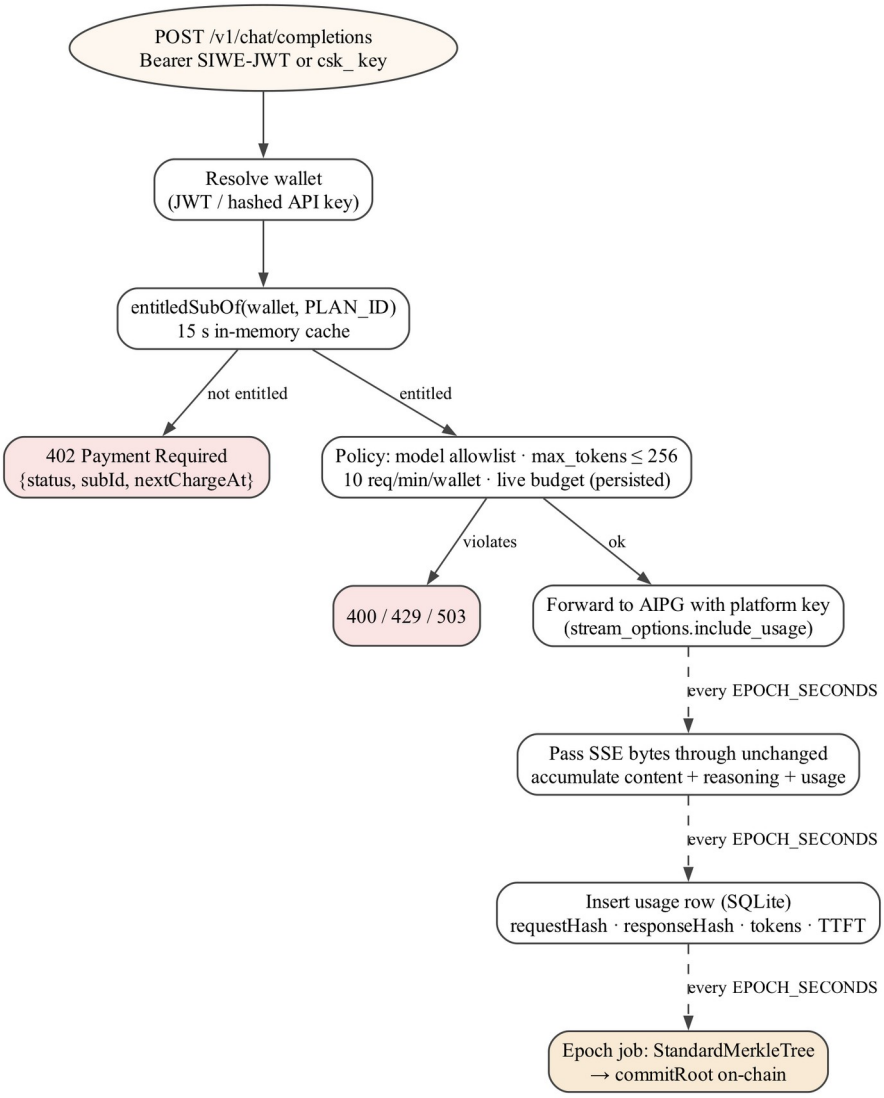


Figure 4.4 Gateway request pipeline

4.8 METERING AND USAGE COMMITMENTS

Each request produces one usage row. Its leaf in the Merkle tree encodes nine fields with the Solidity ABI types shown in Table 4.2. `requestHash` is the Keccak-256 hash of the forwarded request serialised as canonical JSON (keys sorted). `responseHash` is the hash of the generated text, including any reasoning the model streamed. Every `EPOCH_SECONDS` (300 s in the demo) the gateway closes the epoch, builds a StandardMerkleTree over its rows, stores the tree, and calls `commitRoot(planId, epoch, root, rowCount, totalTokens, uri)`. Empty epochs are skipped. Anyone can later fetch a row's proof from the gateway and check it on-chain. The registry also exposes `hashRow`, so the leaf can be recomputed from the row's fields on-chain rather than trusted from the gateway (Figure 4.5).

Table 4.2 Usage-row leaf encoding

Field	ABI type	Meaning
epoch	uint64	floor(timestamp / EPOCH_SECONDS)
wallet	address	Authenticated wallet
subId	uint256	Entitled subscription that paid for the request
model	string	Requested model, e.g. qwen3:8b
promptTokens, completionTokens	uint32, uint32	Usage reported by the upstream
requestHash, responseHash	bytes32, bytes32	Keccak-256 of request and response
timestamp	uint64	Completion time (seconds)

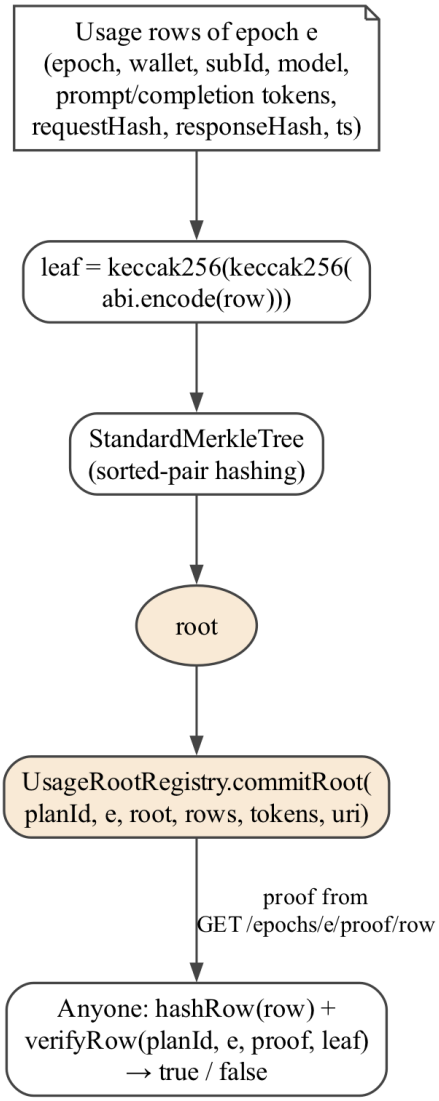


Figure 4.5 Per-epoch usage commitment and verification

4.9 ONE BILLING PERIOD END TO END

Figure 4.6 shows the messages exchanged in one period. The subscriber signs exactly once, at enrolment. Every later period is opened by a keeper's charge and closed by the gateway's usage root, with no further action from the subscriber.

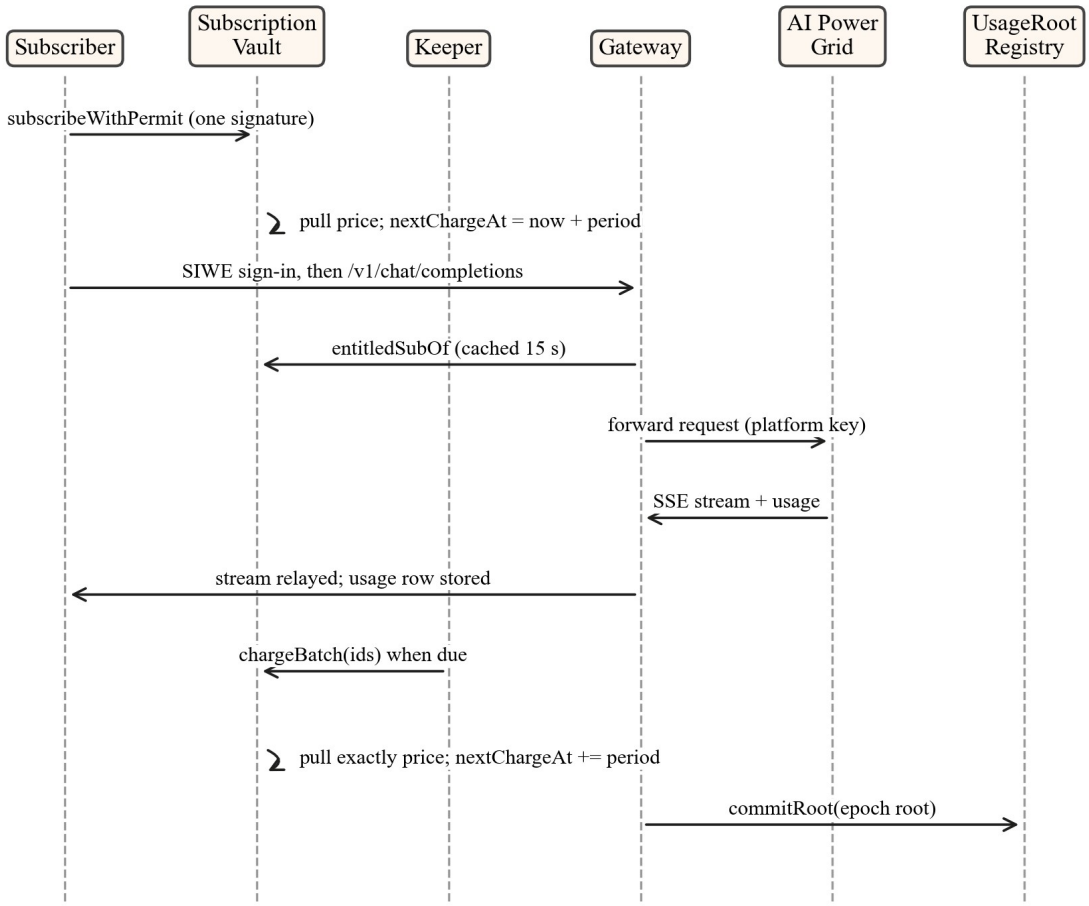


Figure 4.6 Message sequence for one billing period

4.10 SAFETY PROPERTIES

The design's guarantees are stated as five invariants and five “keeper cannot” properties, each mapped to a named test (Chapter 6).

Table 4.3 Protocol invariants

Name	Statement
I1 noOvercharge	For every subscriber, total pulled ≤ Σ price × (1 + ⌊(now − startedAt) / period⌋) over its subscriptions
I2 noChargeAfterTerminal	No successful charge ever happens after a subscription is Cancelled or Lapsed
I3 solvency	Vault token balance = Σ claimable over all accounts
I4 anchoredSchedule	nextChargeAt = startedAt + period × chargeCount for every subscription
I5 ownFundsOnly	No account ever withdraws more than it was credited

Table 4.4 What a keeper (or any caller) cannot do

Attack	Why it fails
Charge early	charge reverts NotDue() while now < nextChargeAt
Charge twice in a period	A successful charge moves nextChargeAt one period ahead, so a second call reverts NotDue()
Charge a cancelled subscription	Terminal status: NotChargeable(Cancelled)
Pull more than the price	The vault has no amount parameter. A direct transferFrom by the attacker fails with ERC20InsufficientAllowance, because the subscriber approved the vault, not the attacker
Redirect the payee's funds	withdraw pays only msg.sender's own balance (NothingToWithdraw), and neither contract has a payee setter

4.11 DESIGN DECISIONS

Table 4.5 Design decisions and their rationale

Decision	Standard / prior art	Reason
Settle in a USD stablecoin	ERC-20; Mita et al. (2019); Pernice et al. (2019)	A 20 USD plan is meaningless in a volatile unit
One-signature enrolment	EIP-712; ERC-2612	The subscriber signs readable, typed terms once
Per-period cap inside the contract	ERC-948 discussion (Owocki, 2018)	A standing allowance alone can be drained in one call
Open, paid trigger	Chainlink 2.0 (Breidenbach et al., 2021)	Supplies the missing clock without granting authority over funds
Batched charging	BatPay (Mayer et al., 2020)	Amortises the fixed cost per transaction
Anchored schedule; grace < period	This work	Late keepers do not shift billing dates, and unpaid subscriptions lapse instead of accruing debt
Failures as state transitions	This work	A batch never reverts on one subscriber, and keepers are not paid for failures
Merkle root per epoch	Merkle (1988); OpenZeppelin (2024)	Verifiable usage at one transaction per epoch
Discrete periods, not streams	Meng & Feng (2025)	Access is sold per period. Streaming complicates entitlement checks

Chapter 5

Implementation

5.1 REPOSITORY STRUCTURE

The implementation is a single pnpm monorepo. Contracts are a Foundry project, and every off-chain package imports contract ABIs and deployed addresses from one generated package, so no address is hard-coded anywhere else. Listing 5.1 shows the layout.

Listing 5.1 Repository layout

```
cashew/
├─ contracts/            Foundry project
│  ├─ src/               MockUSDC, PlanRegistry, SubscriptionVault, UsageRootRegistry
│  ├─ test/              unit, fuzz and invariant (handler-based) tests
│  ├─ script/            Deploy.s.sol, Seed.s.sol
│  └─ deployments/       <chainId>.json written by the deploy scripts
├─ packages/shared/      generated ABIs + addresses.json keyed by chainId (31337,
11155111, 84532)
├─ keeper/               permissionless keeper service (Node.js, TypeScript, viem)
├─ gateway/              entitlement gateway, metering, epoch roots (Express, SQLite)
├─ dashboard/            Next.js 14 App Router + wagmi + RainbowKit + Tailwind
├─ sim/                  experiments and benchmarks; plots.py
├─ results/              machine-written results (JSON) and RESULTS.md
└─ scripts/              local-up, deploy-testnet, testnet-up orchestrators
```

5.2 SMART CONTRACTS

5.2.1 MOCKUSDC

The settlement token extends OpenZeppelin's ERC20 and ERC20Permit with six decimals, like USDC. A `mint` function open to anyone, capped at 1,000 tokens per call, serves as the testnet faucet for the dashboard's “Get test mUSDC” button.

5.2.2 PLANREGISTRY

Plans are stored in an array with ids starting at 1, so 0 can mean “none”. `createPlan` validates its inputs with custom errors (`ZeroAddress`, `ZeroPrice`, `ZeroPeriod`, `GraceNotLessThanPeriod`) and emits `PlanCreated`. The only mutator is `setPlanActive`, restricted to the plan's provider. A `getPlanTerms` view returns the economic fields without the metadata string, so the vault can read terms without copying a string across contracts.

5.2.3 SUBSCRIPTIONVAULT

The vault implements the algorithm of Section 4.4 using OpenZeppelin's `SafeERC20` and `ReentrancyGuard`. Two implementation choices matter for cost. Because plan terms are immutable, the vault caches them on first use in its own storage (two packed slots) instead of calling the registry on every charge. And all status checks and schedule updates happen before the external `transferFrom`, following the checks-effects-interactions order. Errors are custom errors carrying their context, for example `NotChargeable(Status)` and `AlreadySubscribed(subId)`. This is what lets the dashboard's attack panel decode and display the exact reason for each revert. A wallet may hold at most one live subscription per plan. `latestSubOf` exposes the most recent one so the gateway can report its status even when it is no longer entitled.

Listing 5.2 Front-run-tolerant enrolment (excerpt of SubscriptionVault.sol)

```
function subscribeWithPermit(uint256 planId, uint256 permitValue, uint256 deadline,
                             uint8 v, bytes32 r, bytes32 s) external nonReentrant
returns (uint256) {
    Terms memory t = _loadTerms(planId);
    // A permit copied from the mempool and submitted first must not block enrolment.
    try IERC20Permit(t.token).permit(msg.sender, address(this), permitValue, deadline,
v, r, s) {} catch {}
    return _subscribe(planId, t);
}
```

5.2.4 USAGEROOTREGISTRY

This Ownable registry keeps an allow-list of committers and stores one record per (plan, epoch): root, row count, total tokens, commit time, committer and a URI where the tree can be fetched. A second commit for the same epoch reverts with `RootAlreadyCommitted`. `verifyRow` checks a proof with OpenZeppelin's `MerkleProof`, and `hashRow` recomputes the StandardMerkleTree leaf of a usage row on-chain.

5.3 DEPLOYMENT

`Deploy.s.sol` deploys the four contracts with a keeper fee of 25 basis points, authorises the gateway's address as root committer, and writes the addresses and start block to `deployments/<chainId>.json`. `Seed.s.sol` creates the two plans used throughout the project and mints test tokens to the subscriber wallets. The **demo plan** costs 20 mUSDC with a 180-second period and 60-second grace, so a lifecycle can be shown live in minutes. The **production-shaped plan** costs 20 mUSDC with a 30-day

period and 3-day grace. A sync script then regenerates the shared ABI and address package.

The contracts were deployed to **Ethereum Sepolia** and source-verified on Etherscan (Figure 5.1). Deployment showed that Sepolia's current fork prices contract creation far above the cancun schedule that local simulation assumes. Creation receipts were 6.53 M gas for MockUSDC and 11.65 M gas for SubscriptionVault. Forge's locally simulated gas limits therefore ran out of gas twice. The deploy script now passes `--skip-simulation`, so gas limits come from the node's own `eth_estimateGas`. The whole deployment and seeding cost about 0.0000000006 ETH at the prevailing gas price of roughly 19 wei.

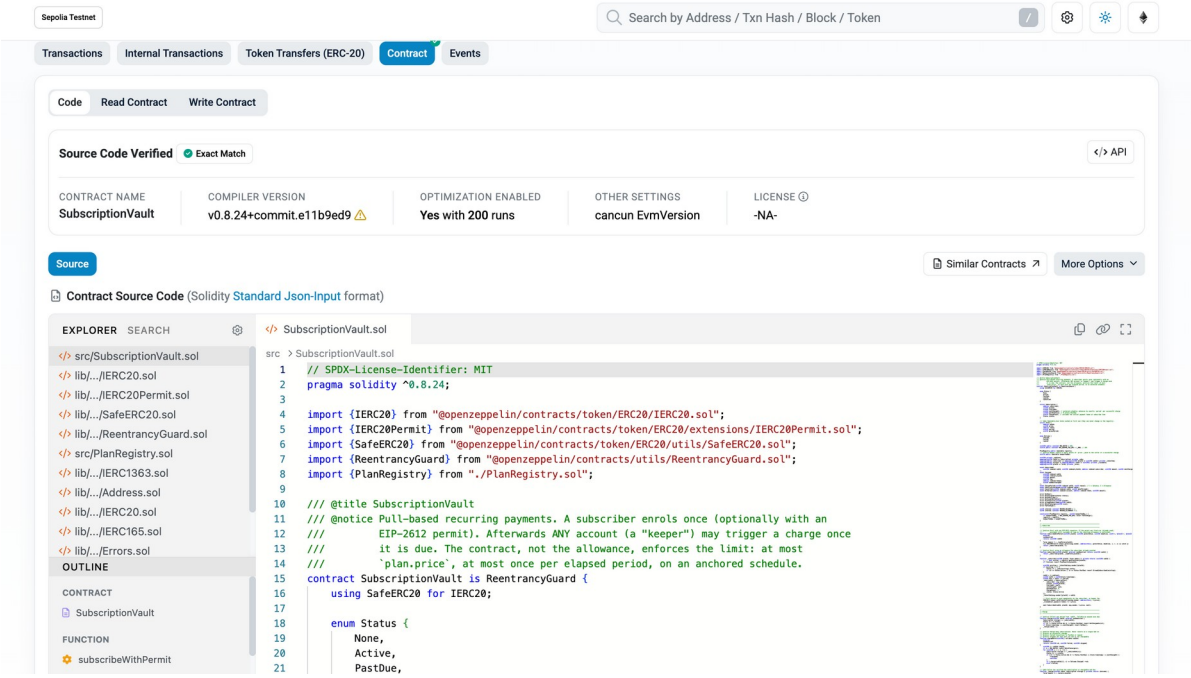


Figure 5.1 SubscriptionVault source verified on Sepolia Etherscan (exact match)

5.4 KEEPER SERVICE

The keeper is a class around two viem clients, one for reads and one for signing. Its `tick()` method implements the loop of Figure 4.3, and a small HTTP server exposes `/health` with the last tick, the current queue size and the keeper's own claimable fee balance. Every processed subscription produces one JSON line: scheduled time, block time of execution, drift, outcome, transaction hash, gas used and batch size. On SIGTERM or SIGINT the keeper finishes the transaction in flight and exits cleanly. The outage experiment in Chapter 6 depends on this. Configuration (RPC URL, chain

id, key, tick interval, batch size) comes from environment variables. The tick is 15 seconds for the demo profile and 6 hours for the production profile.

5.5 CASHEW GATEWAY

The gateway is an Express application whose endpoints are listed in Table 5.1. SIWE messages are parsed and verified with viem's EIP-4361 implementation: domain, chain id, single-use nonce and signature are all checked. The usage store is a single SQLite file holding usage rows, hashed API keys, committed epochs and the persisted live-request counter. In live mode the gateway enforces the caps described in Section 4.7. In mock mode it never touches the network.

Table 5.1 Gateway HTTP endpoints

Endpoint	Purpose
GET /auth/nonce, POST /auth/verify	Sign-In With Ethereum → one-hour JWT bound to the wallet
POST /keys	Mint a csk_ API key for the signed-in wallet (JWT only; only the hash is stored)
POST /v1/chat/completions	OpenAI-compatible chat, streaming or not; 402 when not entitled
GET /v1/models	Allow-listed models, enriched with availability from AIPG in live mode
GET /epochs, /epochs/:e/rows, /epochs/:e/tree	Committed epochs, their rows and the stored Merkle tree
GET /epochs/:e/proof/:rowId	Leaf values, leaf hash and Merkle proof for one usage row
GET /health	Mode, remaining live budget, last committed epoch, cache size

5.6 INTEGRATION WITH AI POWER GRID

The project uses AIPG's public hosted API, not a self-hosted coordinator. To make the grid's supply side part of the demonstration, the team pulled the open-weight `qwen3` models with Ollama on its own GPU and ran the AIPG worker binary. The worker's setup wizard was used to bind the worker to the team's Grid account. The coordinator confirms a worker by routing a canary job to it, after which `GET /v1/workers` lists it as online. The worker list taken during the evaluation (Chapter 6) shows the team's qwen3 workers online alongside community workers. On the demand side, the gateway holds a single platform API key and meters usage per subscriber itself, because AIPG keys are per account rather than per end user.

5.7 DASHBOARD

The dashboard is a Next.js application using wagmi and RainbowKit for wallet connection. It reads all contract state directly from the chain through the shared ABIs. Every transaction hash and address links to the block explorer of the connected chain. The screenshots in this section were captured from the dashboard running against the Sepolia deployment, with real transactions.

5.7.1 OVERVIEW AND PROVIDER PAGES

The overview page (Figure 5.2) shows the deployed contract addresses, the live health of the keeper and the gateway, live counters derived from on-chain events, and the list of plans. The provider page (Figure 5.3) creates plans, lists the provider's own plans with subscriber counts, opens or closes them, and withdraws revenue.

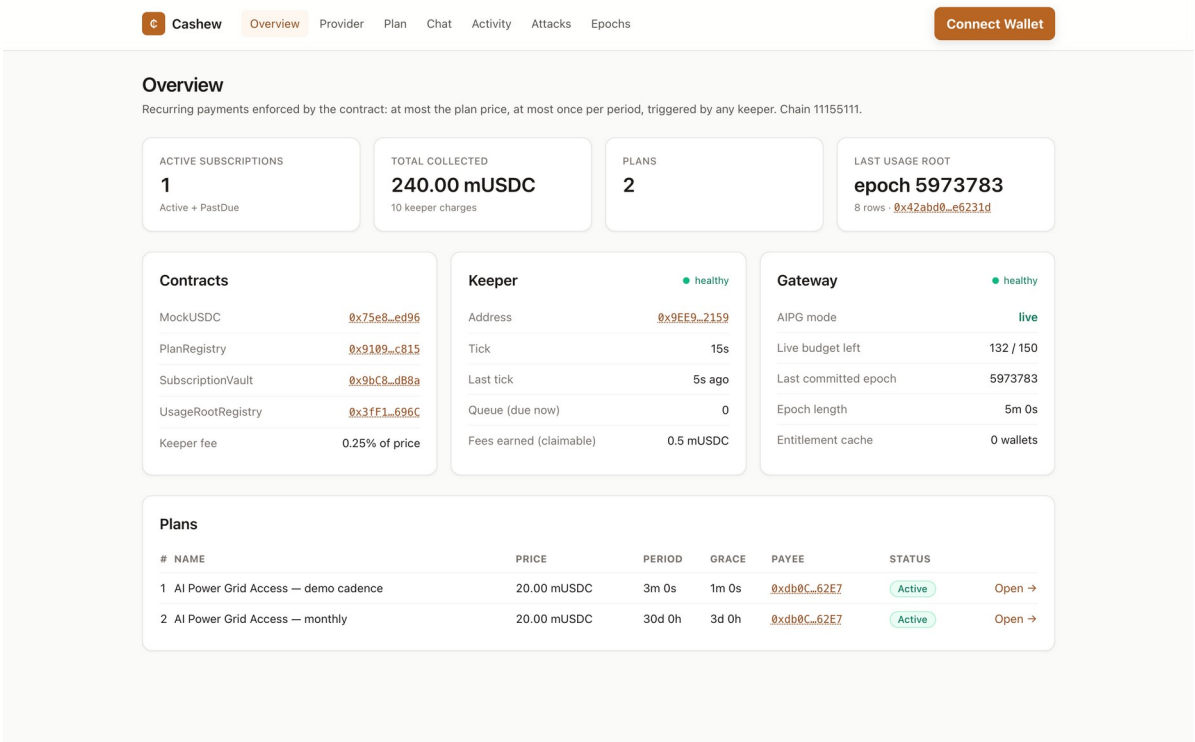


Figure 5.2 Overview page on Sepolia: contracts, keeper and gateway health, counters and plans

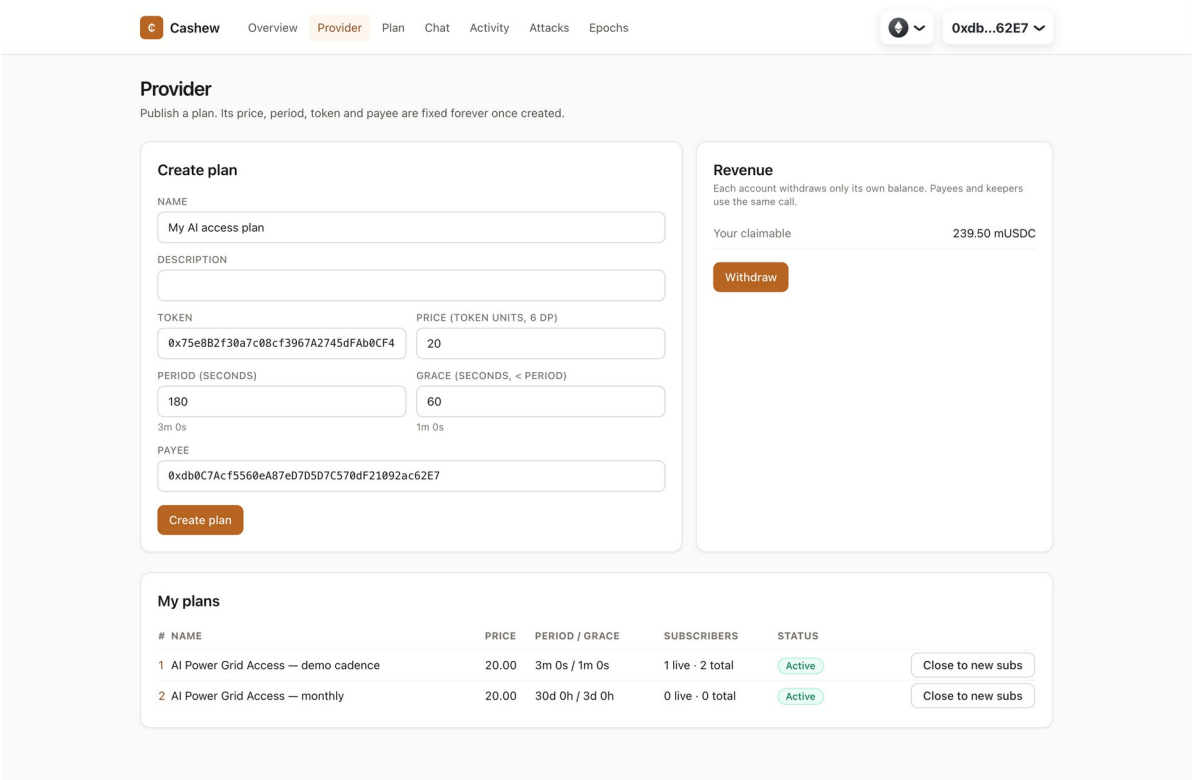


Figure 5.3 Provider page: plan creation, own plans and revenue withdrawal

5.7.2 PLAN AND SUBSCRIPTION PAGE

The plan page (Figure 5.4) shows the immutable terms. It lets a subscriber mint test tokens and enrol with one permit signature and one transaction, and shows the live subscription card: status, entitlement, a countdown to the next charge, charge count and remaining allowance cycles. It also carries the cancel, re-approve and (for demonstrations) drain-balance actions.

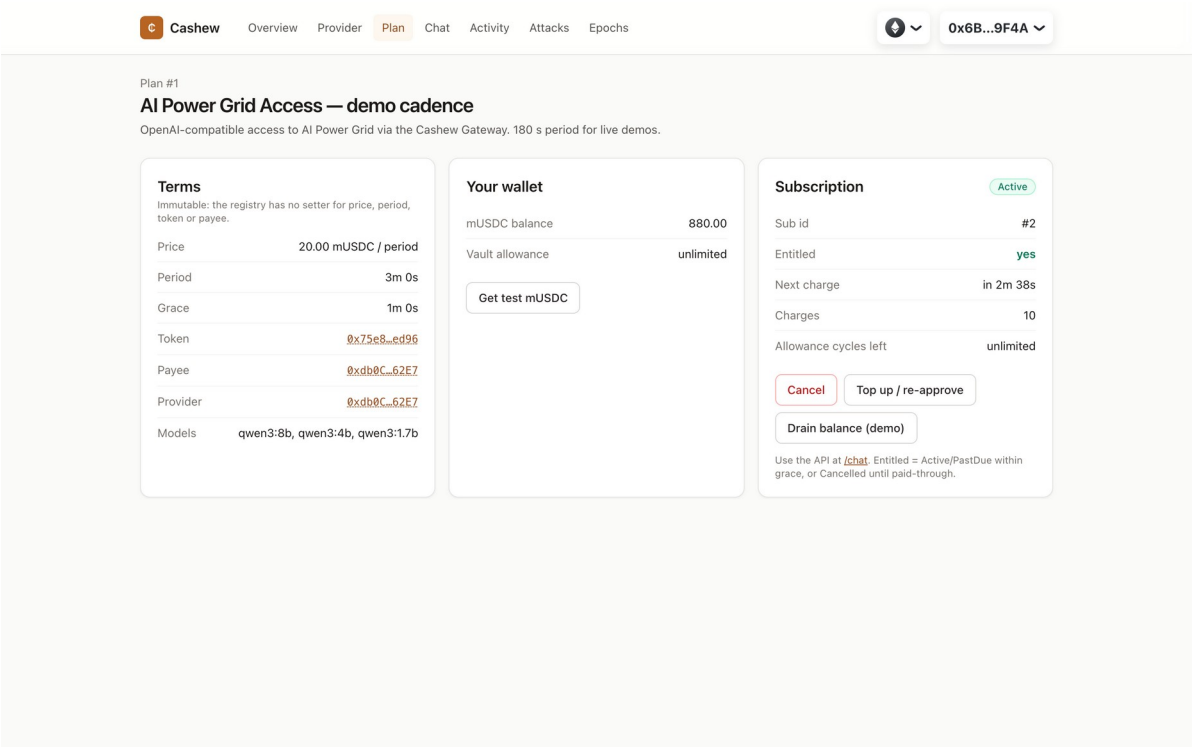


Figure 5.4 Plan page with an active, entitled subscription after ten charges

5.7.3 CHAT PAGE AND THE 402 PATH

The chat page signs in with SIWE, lists the allow-listed models, and streams a completion through the gateway, showing time to first token, total time and token counts under each reply (Figure 5.5). qwen3 streams its reasoning separately from its answer, and the page renders the reasoning in a muted block. When the connected wallet has no entitled subscription, the gateway's 402 response is rendered as a banner carrying the on-chain status and a link to the plan (Figure 5.6).

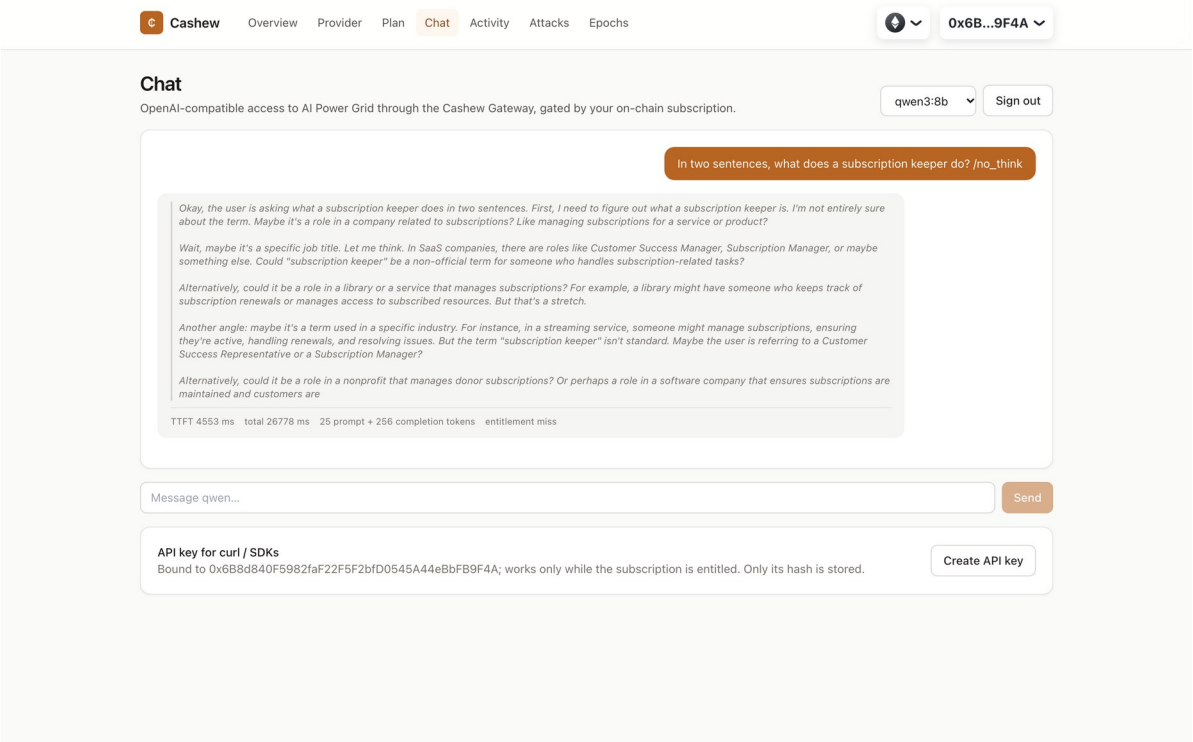


Figure 5.5 Live chat with qwen3:8b on AI Power Grid through the gateway (reasoning shown muted)

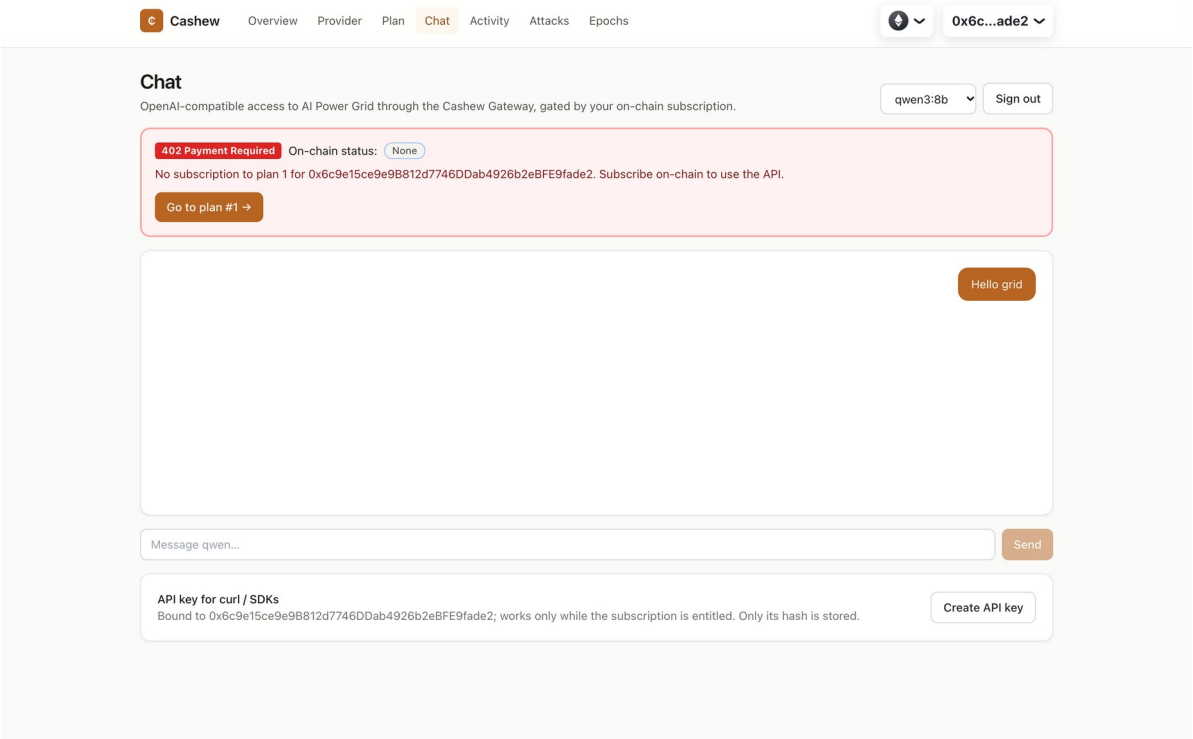


Figure 5.6 HTTP 402 Payment Required for a wallet with no subscription

5.7.4 ACTIVITY, ATTACK PANEL AND EPOCH VERIFIER

The activity page (Figure 5.7) is a live feed of protocol events, showing each keeper charge's drift from its scheduled time. The attack panel (Figure 5.8) runs the five

keeper attacks of Table 4.4 from an outsider wallet. Each attack is simulated against the chain and the decoded custom error is displayed. An optional switch broadcasts the reverting transaction so it can be inspected on the explorer. The epoch verifier is described with the results in Chapter 6.

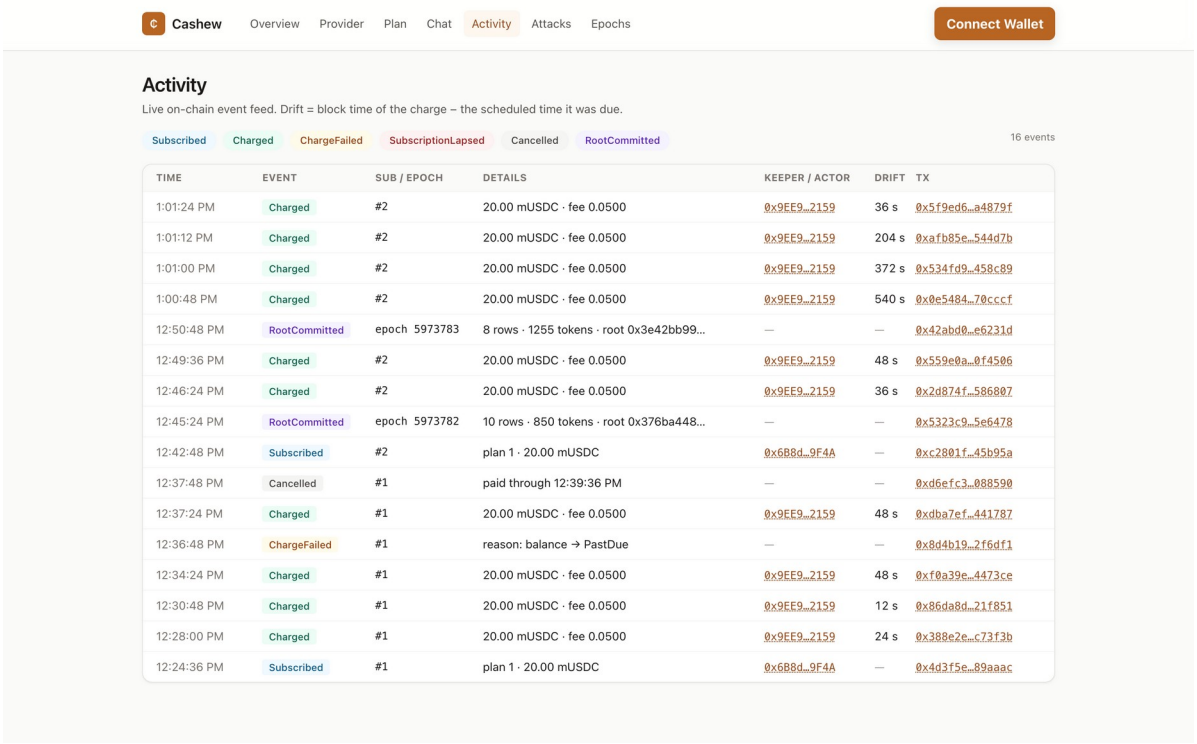


Figure 5.7 Activity feed: subscriptions, keeper charges with drift, failure, cancellation and usage roots

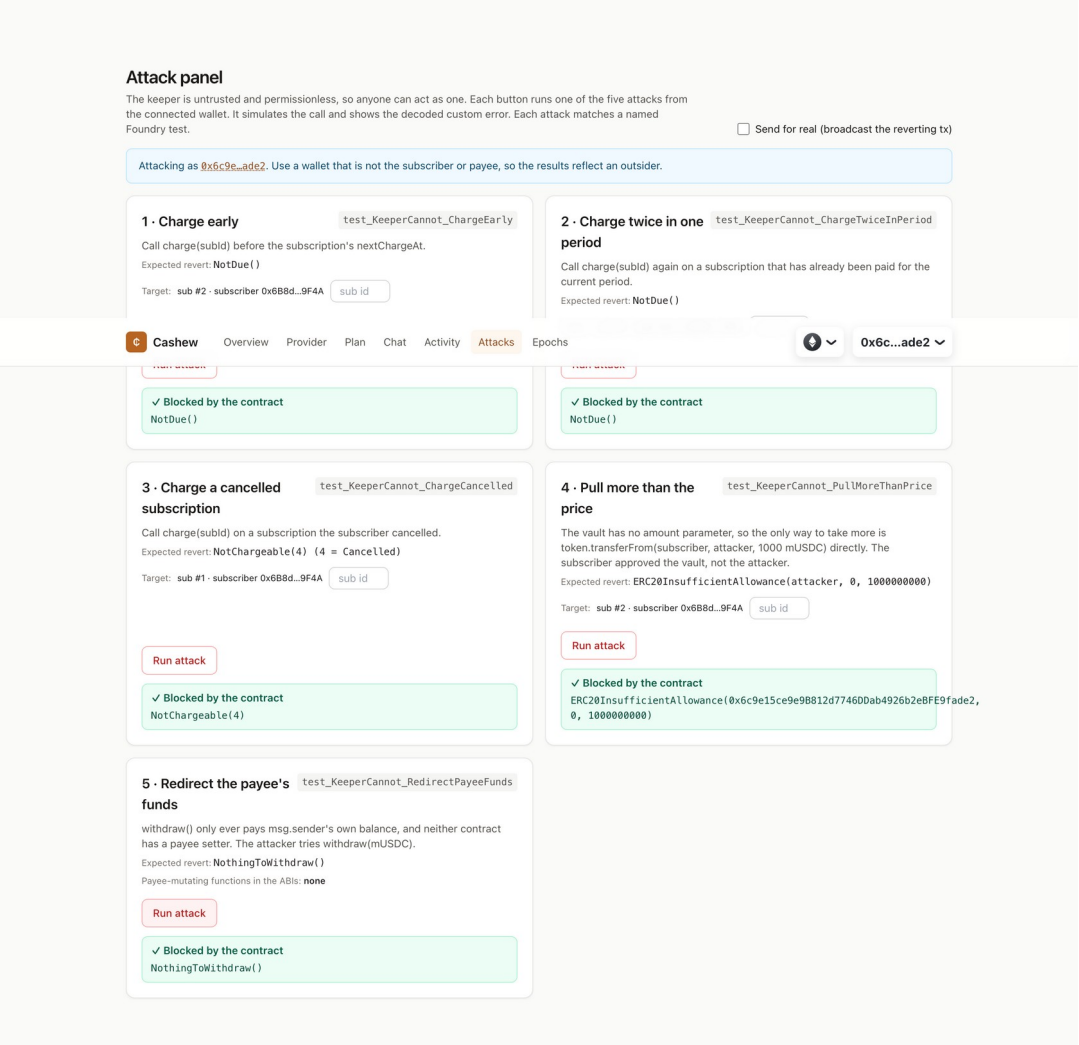


Figure 5.8 Attack panel: all five attacks rejected by the contract with decoded errors

5.8 LOCAL DEVELOPMENT STACK

`pnpm local:up` starts a fresh anvil chain with one-second blocks, deploys and seeds the contracts with the public anvil test keys, regenerates the shared package, and starts the keeper, the gateway in mock mode and the dashboard. For the local demonstration the dashboard offers anvil-only burner wallets, so the full flow runs without a browser extension. These wallets are disabled for public networks. The same orchestration pattern (`pnpm testnet:up`) runs the keeper, the gateway and the dashboard against Sepolia.

Chapter 6

Testing and Results

6.1 EVALUATION METHODOLOGY

Every number in this chapter was produced by a script in the repository that ran during the project. The scripts write machine-readable files to `results/`, and a summariser renders them into `results/RESULTS.md`. Quantities that could not be measured are reported as such rather than estimated. The evaluation has four parts: (1) correctness, through unit, fuzz and invariant tests of the contracts and end-to-end tests of the gateway; (2) cost, through gas measurements and a cost model; (3) timeliness and liveness, through a 500-subscriber schedule simulation and a keeper-outage experiment on a local chain with time travel; and (4) behaviour in the real environment, through a scripted lifecycle on Ethereum Sepolia and live inference on AI Power Grid.

6.2 CONTRACT TEST SUITE

The Foundry suite contains one unit-test file per contract, a fuzz-test file and a handler-based invariant campaign. Table 6.1 summarises it. Unit tests cover every revert path and every state transition, including the permit front-run fallback, duplicate-subscription rejection, inactive plans, the grace-less-than-period check and zero withdrawals. The three fuzz properties run 1,000 random cases each. The invariant campaign drives a handler through random sequences of subscribe, charge, batch charge, keeper sweep, cancel, time warp, drain, top-up, allowance change and withdrawal across three plans (including one whose grace is one second short of its period). It checks the five invariants of Table 4.3 after every call, using ghost accounting derived from token-balance movements rather than from the vault's own bookkeeping.

Table 6.1 Contract test summary (forge test)

Kind	Tests	Passed	Configuration
Unit	68	68	Every revert and state transition; five KeeperCannot tests
Fuzz	3	3	1,000 runs each
Invariant predicates	5	5	256 runs × depth 64 = 16,384 handler calls
Total	72 (+5 predicates)	all	forge coverage: 100 % lines, statements, branches and functions of src/

Table 6.2 Fuzz properties

Test	Property checked
testFuzz_ChargedAmountEqualsPrice	For random price, period and lateness, one charge moves exactly price from the subscriber, and the keeper and payee are credited fee and price − fee
testFuzz_ChargeBeforeDueReverts	For any caller and any time before nextChargeAt, charge reverts NotDue()
testFuzz_AnchoredSchedule	After k charges with random delays within grace, nextChargeAt = startedAt + (k + 1)·period

Table 6.3 Results of the five keeper-attack tests

Test	Result
test_KeeperCannot_ChargeEarly	pass: reverts NotDue()
test_KeeperCannot_ChargeTwiceInPeriod	pass: second call reverts NotDue(); subscriber charged once
test_KeeperCannot_ChargeCancelled	pass: reverts NotChargeable(Cancelled) even ten periods later
test_KeeperCannot_PullMoreThanPrice	pass: direct transferFrom reverts ERC20InsufficientAllowance; a charge moves exactly price
test_KeeperCannot_RedirectPayeeFunds	pass: attacker's withdraw reverts NothingToWithdraw; no payee setter exists

Building the invariant suite found one defect, in the test harness rather than the contract. The handler's `vm.prank` was consumed by a view call that came before the charge, so fees were credited to the handler contract and I3 and I5 failed. Moving the prank directly in front of the charge fixed it. Ten gateway end-to-end tests run against a local chain: SIWE sign-in and single-use nonces, 402 for unsubscribed wallets, permit enrolment, streaming and non-streaming completions with usage rows, the model allowlist, API-key minting rules, epoch commitment with on-chain `verifyRow`, and equality of the JavaScript leaf with Solidity's `hashRow`. All ten pass.

6.3 GAS COST PER OPERATION

Table 6.4 and Figure 6.1 give the execution gas of each operation as measured by Foundry across the unit tests. Ranges reflect different paths. The minimum of `charge`, for example, is a reverting call, and its maximum includes a first-time keeper credit. Foundry's figures exclude the 21,000 intrinsic gas and calldata cost of a real transaction, which are included in the receipt-based measurements that follow.

Table 6.4 Execution gas per operation (Foundry gas report, unit tests)

Operation	Min	Median	Max
createPlan	23,226	161,973	161,973
subscribeWithPermit	213,051	253,892	290,419
subscribe	36,446	241,640	241,640
charge	29,057	46,360	94,772
chargeBatch	27,118	49,704	118,593
cancel	23,762	30,458	30,498
withdraw	29,149	48,111	67,074
commitRoot	25,044	27,397	118,440

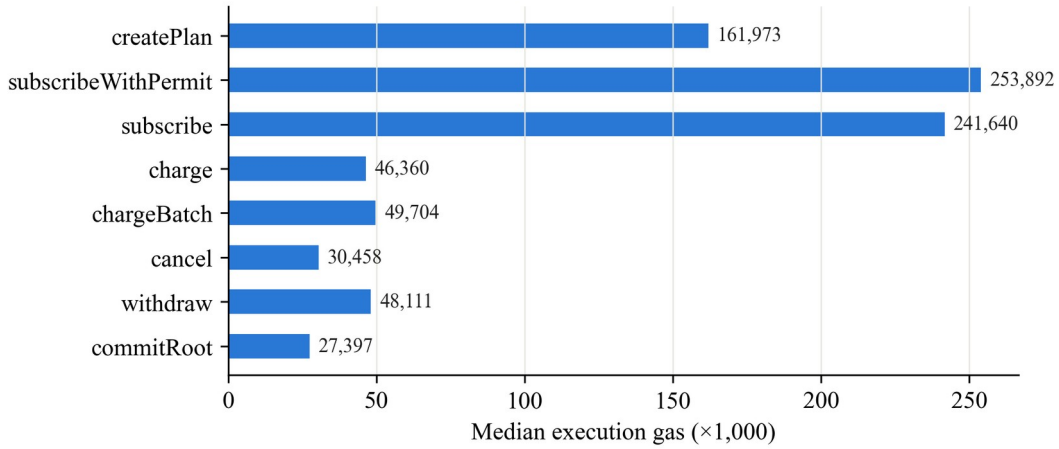


Figure 6.1 Median execution gas per operation

6.4 BATCH CHARGING

To measure what a keeper actually pays, real transactions were sent on a scratch chain with 200 funded subscribers and the receipts' `gasUsed` was recorded for batch sizes from 1 to 200 (Table 6.5, Figure 6.2). A single `charge` transaction used 71,595 gas. The very first keeper charge, which turns the keeper's claimable balance from zero to non-zero, used 88,695. Batching amortises the fixed transaction cost. Per-subscriber gas falls from 72,525 at batch size 1 to 27,567 at 200, a 62 % reduction, and the curve is almost flat beyond 50.

Table 6.5 Receipt gas of chargeBatch by batch size

Batch size	Total gas	Gas per subscriber
1	72,525	72,525
5	181,700	36,340
10	318,183	31,818
25	727,701	29,108

Batch size	Total gas	Gas per subscriber
50	1,410,472	28,209
100	2,776,919	27,769
200	5,513,435	27,567

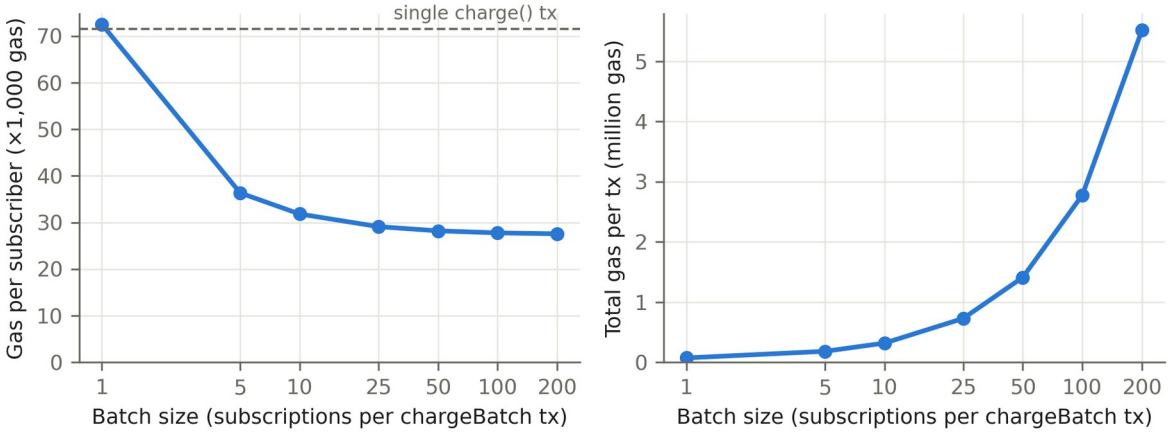


Figure 6.2 Gas per subscriber and total gas per transaction against batch size

6.5 COST MODEL

Gas was converted to money with the Base Sepolia gas price measured at run time, 0.006 gwei at block 48,178,885. The L1 data fee was quoted at the same time by the network's GasPriceOracle. Two inputs are assumptions, not measurements: ETH = 3,000 USD, and a Base-mainnet-like L2 gas price of 0.01 gwei for comparison (that column excludes the L1 data fee). Table 6.6 shows the results. At a batch of 100, keeping one 20 USD subscription current costs about 0.0005 USD, which is 0.0025 % of the cycle and one hundredth of the 0.05 USD keeper reward. Running a keeper is therefore profitable by a wide margin under both gas-price scenarios (Figure 6.3).

Table 6.6 Cost of operations (ETH/USD = 3,000 assumed)

Operation	Gas	USD per subscriber (Base Sepolia, measured)	% of 20 USD cycle	USD per subscriber (assumed mainnet L2)
charge (single tx)	71,595	0.00129	0.0064 %	0.00215
chargeBatch(10)	318,183	0.00057	0.0029 %	0.00095
chargeBatch(50)	1,410,472	0.00051	0.0025 %	0.00085
chargeBatch(100)	2,776,919	0.00050	0.0025 %	0.00083
chargeBatch(200)	5,513,435	0.00050	0.0025 %	0.00083
subscribeWithPermit	274,892	0.00495	0.0247 %	0.00825
cancel	51,458	0.00093	0.0046 %	0.00154
commitRoot (per epoch)	48,397	0.00087	—	0.00145

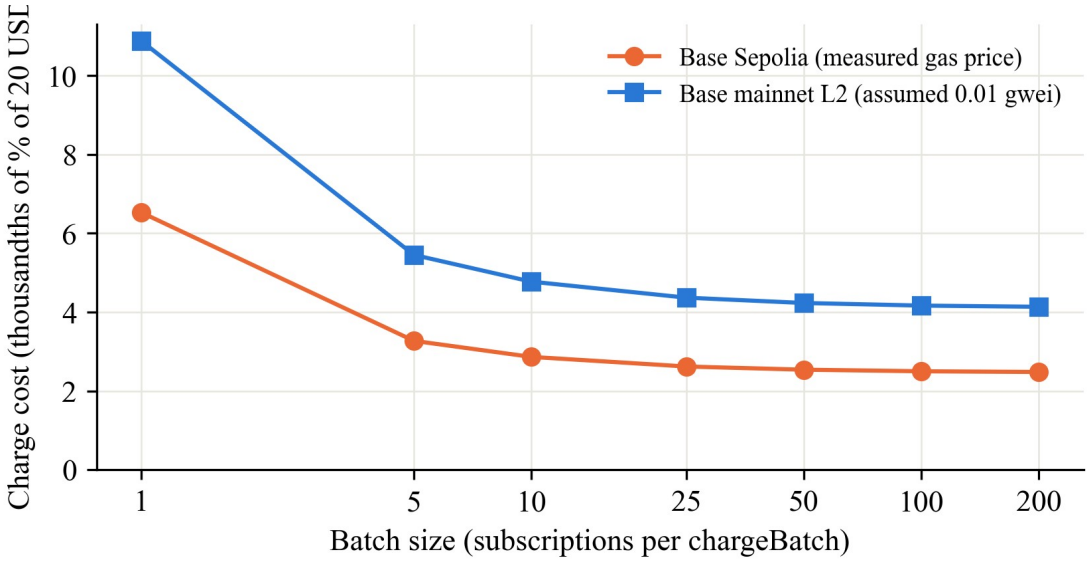


Figure 6.3 Charge cost as a fraction of the 20 USD cycle against batch size

Because the testnet deployment is on Ethereum Sepolia, its own gas price (about 19 wei at deployment) was even lower. The dollar figures above deliberately use the Base network, the intended production target, so they do not depend on the unusually cheap testnet.

6.6 SCHEDULE SIMULATION

Timeliness was evaluated on a scratch chain with time travel. 500 subscribers enrolled in the 30-day plan (grace 3 days), with start times spread across the first period: one per 5,184-second slot, uniformly jittered within the slot from a fixed seed so that due times do not align with any tick grid. The real keeper class then ran at fixed tick intervals of 1 minute, 10 minutes, 1 hour and 6 hours for twelve periods (360 days), with each keeper transaction mined two seconds after its tick, about one Base block. Each interval ran on its own fresh chain with an identical setup. Ticks at which nothing was due were counted but not executed, since nothing would happen on-chain. Within the horizon, 5,500 renewals fall due: eleven keeper renewals per subscriber in addition to the initial payment.

Table 6.7 Schedule simulation: 500 subscribers, 12 × 30-day periods

Tick	Due	Executed	Missed	Lost	Drift p50	Drift p95	Drift max	Keeper txs	Gas / charge
1 min	5,500	5,500	0	0	33 s	59 s	61 s	5,500	72,534
10 min	5,500	5,500	0	0	303 s	570 s	601 s	5,489	72,444

Tick	Due	Executed	Missed	Lost	Drift p50	Drift p95	Drift max	Keeper txs	Gas / charge
1 h	5,500	5,500	0	0	1,978 s	3,444 s	3,589 s	5,082	69,096
6 h	5,500	5,500	0	0	10,669 s	20,336 s	21,583 s	1,320	38,158

Every due renewal was executed at every interval, with none missed or lost. Drift behaves as the design predicts: it is bounded by the tick interval plus the two-second inclusion delay, and its median is close to half the interval (Figure 6.4). Longer intervals also batch more subscriptions per transaction, so gas per charge falls from about 72,500 at one-minute ticks to 38,158 at six-hour ticks. This exposes the operating trade-off: a 6-hour keeper charges up to six hours late but at about half the cost. Since the shortest grace in the production plan is three days, every interval tested keeps subscribers entitled throughout.

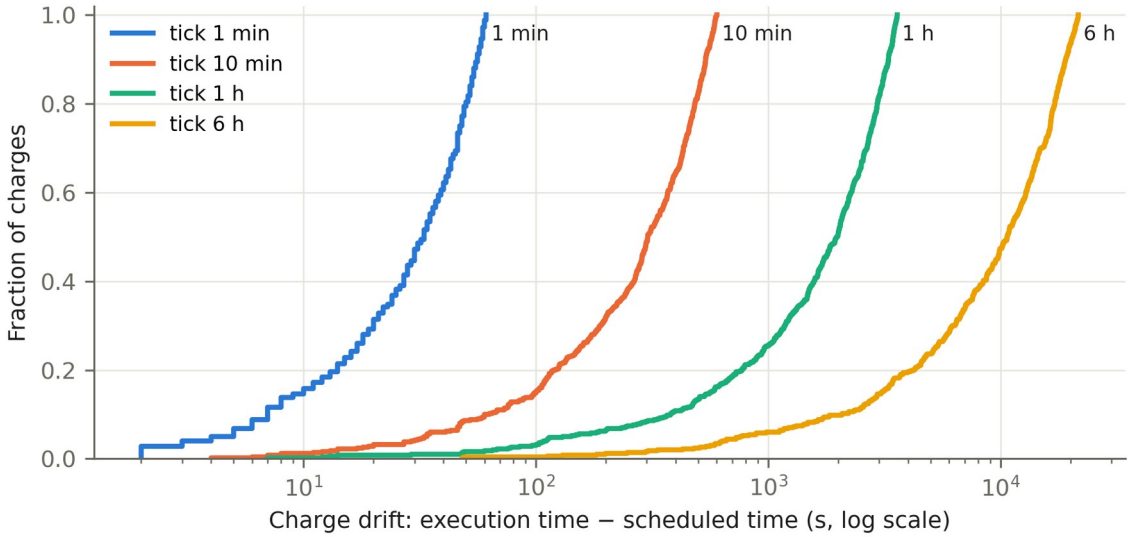


Figure 6.4 Cumulative distribution of charge drift for the four keeper tick intervals

6.7 KEEPER OUTAGE AND RECOVERY

Liveness was tested with the real keeper process rather than the keeper class. Twenty subscribers enrolled in the 180-second demo plan, and the keeper charged everyone once. It was then stopped with SIGTERM, exited cleanly with code 0 after logging its graceful shutdown, and chain time was advanced by 546 seconds: three full periods, well beyond the 60-second grace. No charge occurred during the outage. On restart the keeper recovered all **60 of 60** missed charges in three transactions within about one second of wall time. The maximum drift was 368 seconds. **No charge was lost** (100 expected, 100 executed), no subscription lapsed, and every subscription still

satisfied `nextChargeAt = startedAt + period × chargeCount`. Each recovered charge settled the next anchored slot rather than the restart time, confirming the design's claim that keeper outages make charges late, not lost.

6.8 LIFECYCLE ON ETHEREUM SEPOLIA

A script executed the complete lifecycle on the public testnet against the deployed demo plan, recording each step's transaction hash, block time, gas and resulting on-chain state (Table 6.8, Figure 6.5). The subscriber enrolled with one permit signature. The keeper charged three periods. The subscriber's balance was then drained, and the next charge emitted `ChargeFailed` and moved the subscription to PastDue while it stayed entitled during grace. After a top-up, the next charge recovered it to Active. Finally the subscriber cancelled, remained entitled until the paid-through time and then lost entitlement. Figure 6.6 shows one of the keeper charges on Etherscan, with the 20 mUSDC transfer from the subscriber to the vault.

Table 6.8 Lifecycle executed on Ethereum Sepolia (subscription #1, demo plan)

#	Step	Event	Status after	Entitled	Charges	Gas
1	subscribeWithPermit	Subscribed	Active	yes	1	1,169,819
2	keeper charge	Charged	Active	yes	2	210,352
3	keeper charge	Charged	Active	yes	3	112,432
4	keeper charge	Charged	Active	yes	4	112,432
5	drain balance	—	Active	yes	4	128,922
6	keeper charge	ChargeFailed (balance)	PastDue	yes (grace)	4	45,160
7	top up (mint 100)	—	PastDue	yes (grace)	4	140,615
8	keeper charge	Charged	Active	yes	5	112,472
9	cancel	Cancelled	Cancelled	yes	5	31,658
10	before paid-through	—	Cancelled	yes	5	—
11	after paid-through	—	Cancelled	no	5	—

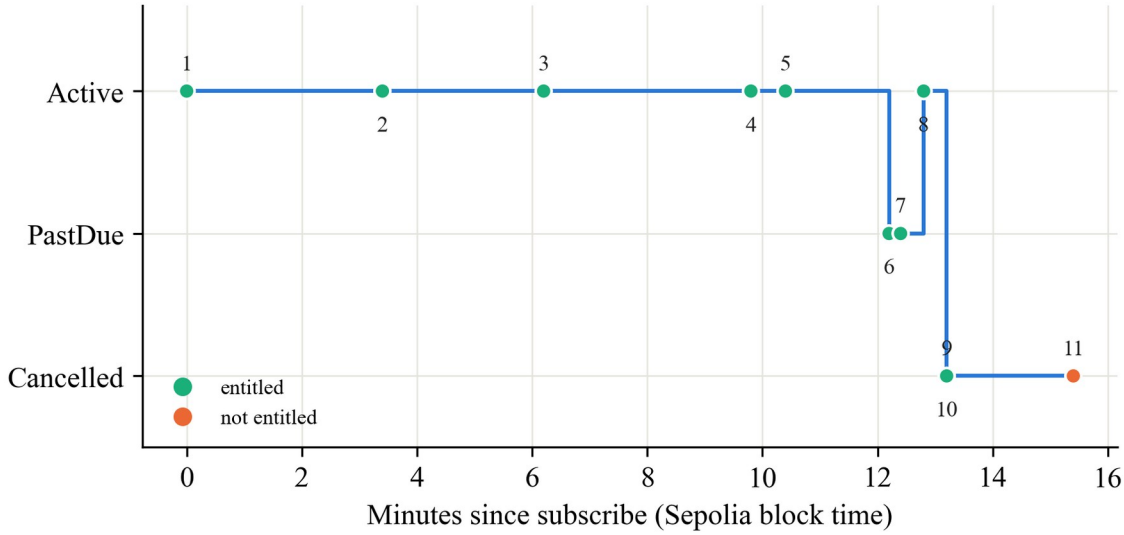


Figure 6.5 Status of subscription #1 over time on Sepolia (numbers refer to Table 6.8)

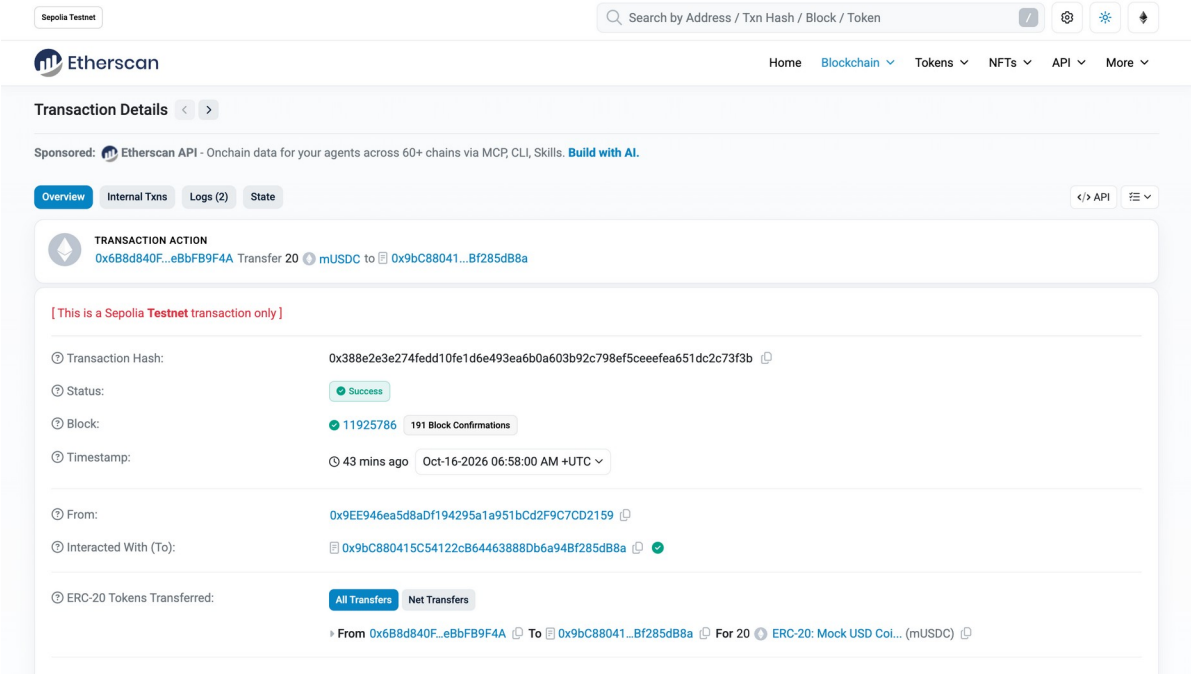


Figure 6.6 A keeper charge on Sepolia Etherscan: 20 mUSDC moved from the subscriber to the vault

The gas figures on Sepolia are higher than on the local chain for operations that create storage: 1.17 M for enrolment against about 0.27 M locally, and 210,352 for a first keeper charge against 88,695. The cause is the same state-creation repricing found during deployment. Repeated charges, which only update existing storage, cost 112,432. Because Base had not adopted that repricing at the time of the measurement, the cost model of Section 6.5 keeps the cancun-priced receipts.

6.9 LIVE INFERENCE THROUGH THE GATEWAY

6.9.1 GRID SNAPSHOT

A snapshot of AIPG's public endpoints, taken immediately before the live runs (no inference credits used), listed 11 workers online. The text models online were qwen3:8b, qwen3:4b and qwen3:1.7b (the team's workers) together with gpt-oss-120b, deepseek-v4-flash-nvfp4 and qwen38-flash-next-uncensored-125b-nvfp4. An earlier snapshot the same morning had shown no qwen3 worker online, which is why the live phase waited until the team's workers reconnected.

6.9.2 LATENCY

With the gateway in live mode on Sepolia, the benchmark sent identical streaming requests (qwen3:8b, max_tokens 64) directly to AIPG and through the gateway, alternating the order within each pair. The total was limited to 30 live requests to protect the shared credit, and the measurement took two runs. The first, 12 pairs, recorded total latency: medians were 5,008 ms direct and 5,053 ms through the gateway. Its time-to-first-token values are missing because the script then counted only `content` tokens, and qwen3 spent all 64 tokens on reasoning. The second run, 3 pairs, counts the first token of any kind (Table 6.9, Figure 6.7).

Table 6.9 Live latency, qwen3:8b, max_tokens 64 (second run, n = 3 per path)

Metric	Direct p50	Direct max	Gateway p50	Gateway max
Time to first token (ms)	4,341	5,456	4,415	4,660
Total (ms)	5,829	6,039	5,006	5,075

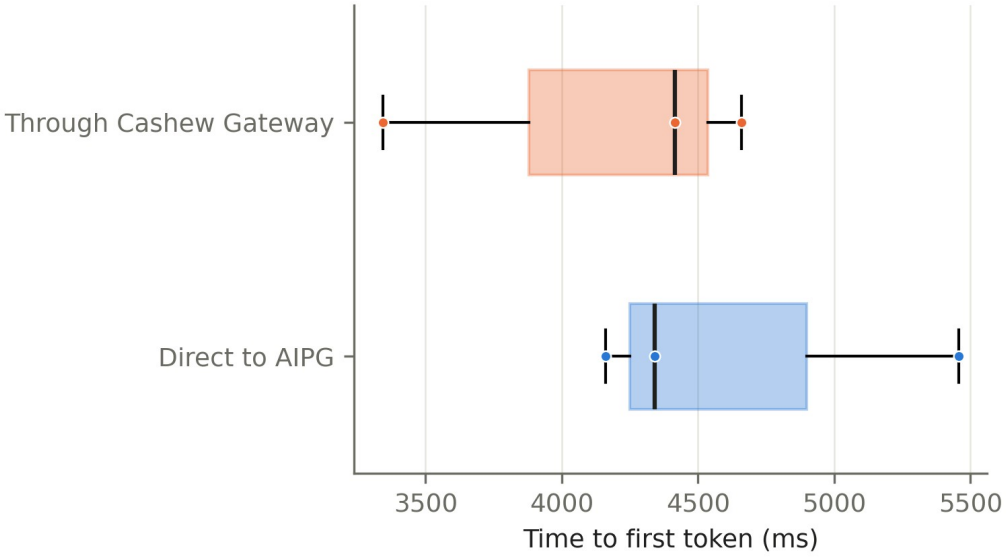


Figure 6.7 Time to first token, direct to AIPG and through the Cashew Gateway

Time to first token is dominated by the grid's queueing and model start-up, about four seconds, which varies by more than a second between consecutive requests. The gateway's own overhead is smaller than that variance. Paired differences (gateway minus direct) had a median of −818 ms for time to first token, and the 12-pair medians of total latency differ by 45 ms, so no gateway overhead can be distinguished from grid variance at this sample size. The on-chain entitlement check, by contrast, was measured directly from the gateway's response headers. Across both runs, uncached checks (three RPC reads against a public Sepolia endpoint) took a median of 442 ms in run one (n = 6) and 600 ms in run two (n = 2), while cached checks took 0.01 ms. The 15-second cache therefore removes the chain from the request path for all but the first request in each window.

6.9.3 METERING AND ON-CHAIN VERIFICATION

The 15 gateway-path benchmark requests and three ordinary chats were metered as 18 usage rows. The gateway committed them in two epochs, 10 rows (850 tokens) and 8 rows (1,255 tokens), with one `commitRoot` transaction each. An exported inclusion proof (length 3) for a row in the latest epoch verified on-chain with `verifyRow = true`. Figure 6.8 shows the dashboard's verifier confirming a row against the committed root, with the leaf recomputed on-chain by `hashRow`. Figure 6.9 shows the same row rejected after one completion token is added, which is the property that makes the commitment useful: the gateway cannot later change what it recorded.

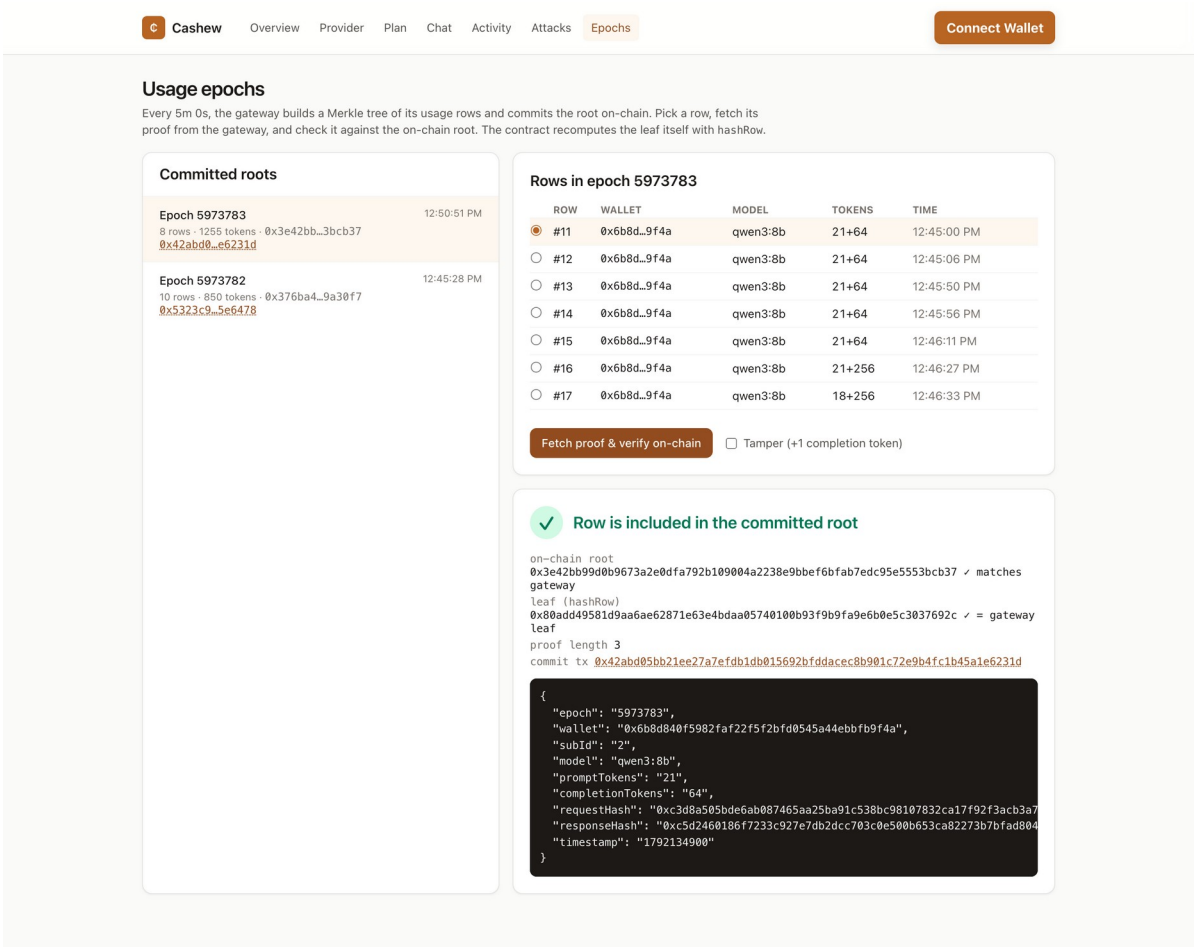


Figure 6.8 Epoch verifier: a usage row proven against the root committed on Sepolia

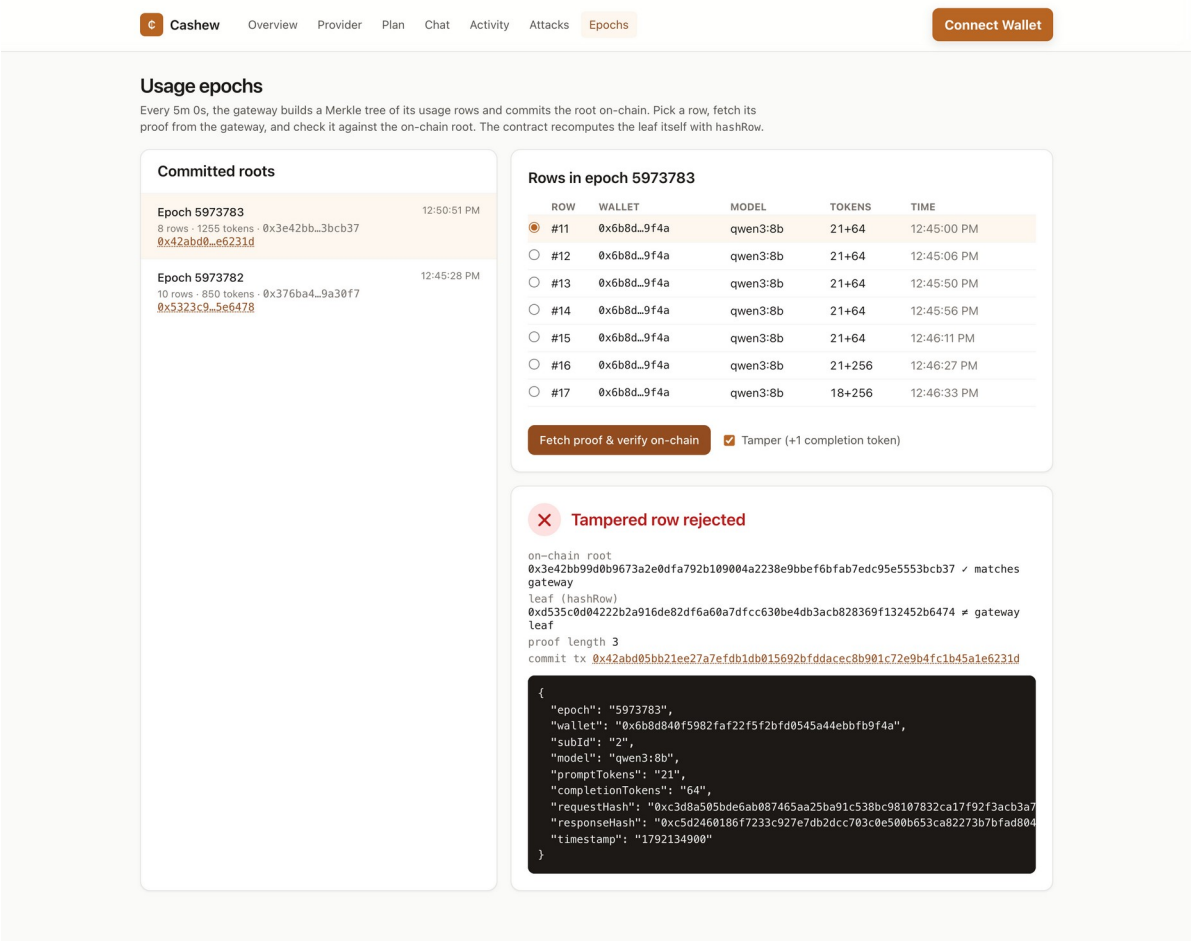


Figure 6.9 Epoch verifier: the same row with one extra completion token is rejected

AIPG did not return a job identifier header during these runs, so the `gridJobId` column of every row is empty. The gateway captures the header when the upstream sends it.

6.10 DISCUSSION

- **Safety.** The five keeper attacks fail in unit tests, in randomised invariant campaigns and against the live Sepolia deployment through the attack panel (Figure 5.8). The protocol's safety does not depend on who runs the keeper.
- **Cost.** Batched charging costs about 27,600 gas per subscriber, which is 0.0025 % of a 20 USD cycle at the measured Base Sepolia price and two orders of magnitude below the keeper's reward. The economics do not depend on gas staying unusually cheap.
- **Timeliness and liveness.** Drift is bounded by the keeper's tick interval, nothing was missed in 22,000 simulated renewals across four intervals, and a three-period outage was fully recovered with nothing lost.

- **Comparison with interim estimates.** Earlier review material quoted indicative figures: 74,830 gas for a single charge, 21,604 per subscriber at a batch of 50, and a 142-second median drift. The measured values reported here replace them: 71,595 gas, 28,209 gas and a median drift equal to about half the keeper's tick interval.

Chapter 7

Conclusion and Future Work

7.1 CONCLUSION

This project set out to give recurring payment a native form on a public blockchain, and to prove it against a real service rather than a toy one. The central idea was to stop asking how a contract can schedule itself and to ask instead how to make it worthwhile for anyone to call it on time, while making the contract safe against whatever that caller attempts. Cashew realises this with three mechanisms that prior designs offered separately but not together: an open trigger, a fee paid by the contract to whoever triggers, and a spending limit of one plan price per elapsed period enforced inside the contract on an anchored schedule.

The objectives were met as follows. The protocol was implemented as four contracts and deployed and source-verified on Ethereum Sepolia. Its safety was established by 72 tests, including fuzz properties and five invariants checked over 16,384 random actions, with full line coverage. The five keeper attacks were also demonstrated failing against the live deployment. Its cost was measured rather than assumed. A batched charge costs about 27,600 gas per subscriber, 0.0025 % of a 20-dollar cycle at the measured Base Sepolia gas price and about one hundredth of the keeper's reward, so the keeper market funds itself. Its timeliness and liveness were shown at scale: 22,000 simulated renewals across four keeper intervals with none missed, drift bounded by the tick interval, and complete recovery after a three-period keeper outage. On the service side, the gateway sold live qwen3 inference from the team's own GPU worker on AI Power Grid only to entitled wallets, returned HTTP 402 to others, and committed its usage to the chain in two epochs whose rows verify on-chain and fail verification when tampered with.

Several results were not obvious at the outset. First, treating payment failure as a state transition rather than a revert matters as much as the cap itself: it lets one batch serve many subscribers without one bad wallet blocking the rest, and it ensures keepers are never paid for failed attempts. Second, the keeper's tick interval is a cost–latency dial rather than a safety parameter. A six-hour keeper is up to six hours late but costs half as much gas per charge, and the grace period, not the keeper, decides when access

ends. Third, the real environment differed from the local one in ways only deployment revealed. Sepolia's state-creation repricing made contract creation about five times more expensive than local simulation predicted, and the live model spent its whole token budget on reasoning, which broke a naive time-to-first-token measurement. Both were caught because results were measured, and both are reported rather than smoothed over.

7.2 LIMITATIONS

The project has clear boundaries, and it is important not to overstate what has been shown.

- **The gateway is a trusted metering party.** A usage root proves that a row is part of what the gateway recorded. It does not prove the record is complete or that the gateway is honest.
- **A single AI Power Grid key** is shared behind the gateway. Per-wallet limits are enforced by the gateway, not by AIPG.
- **Revenue is not yet distributed to individual GPU workers.** The provider's revenue goes to one payee address. Automated, proof-based payouts to workers remain future work.
- **Testnet only, with a mock token.** No real USDC was used and the contracts have not been audited.
- **Linear due-scan.** `dueSubscriptions` scans subscription ids in pages, including terminal ones. That suits the evaluated scale, but a production deployment would want an indexed due-queue.
- **Non-standard tokens.** The vault pre-checks balance and allowance. A token whose transfers revert for other reasons, such as blocklists or pauses, would revert a whole batch, and fee-on-transfer tokens are not supported.
- **Small live latency sample.** Live latency was limited to 30 requests to protect the shared credit, so time-to-first-token statistics rest on three pairs. They show grid variance dominating, but they cannot resolve the gateway's own overhead.

7.3 FUTURE WORK

- **Worker payouts from usage roots.** Allocate each period's revenue to worker addresses pro rata to the tokens they served, claimable with a Merkle proof against the same epoch root. This closes the loop between subscribers and GPU owners.
- **Reducing trust in metering.** Let independent keepers or validators attest usage, for example by co-signing epoch roots or sampling rows against grid job records, so the gateway is no longer the only witness.
- **Gasless enrolment.** Use an ERC-4337 smart account with a paymaster so that a subscriber holding only stablecoins can enrol without any native token.
- **Production hardening.** An indexed due-queue, support for additional settlement tokens, an external security audit, and deployment on Base, where the cost model already applies.
- **Standardisation.** Package the vault's interface (subscribe, charge, chargeBatch, cancel, isEntitled) as a reusable standard, so that other services can sell access with the same safety guarantees.

In short, Cashew shows that a subscription on a public chain can be automatic, capped and revocable without trusting the party that triggers it. A real, independently operated compute network can then be sold through it, with what was consumed committed on-chain where anyone can check it.

References

Abadi, A., Murdoch, S. J., & Zacharias, T. (2023). Recurring contingent service payment. In *2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P)*. IEEE.

AI Power Grid. (2026). *AI Power Grid documentation* [Online documentation]. Retrieved October 16, 2026, from https://aipowergrid.io/docs

Antonopoulos, A. M., & Wood, G. (2018). *Mastering Ethereum: Building smart contracts and DApps*. O'Reilly Media.

Atzei, N., Bartoletti, M., & Cimoli, T. (2017). A survey of attacks on Ethereum smart contracts (SoK). In *Principles of Security and Trust (POST 2017)* (Lecture Notes in Computer Science, Vol. 10204, pp. 164–186). Springer.

Bloemen, R., Logvinov, L., & Evans, J. (2017). *EIP-712: Typed structured data hashing and signing* (Ethereum Improvement Proposal No. 712). https://eips.ethereum.org/EIPS/eip-712

Breidenbach, L., Cachin, C., Chan, B., Coventry, A., Ellis, S., Juels, A., Koushanfar, F., Miller, A., Magauran, B., Moroz, D., Nazarov, S., Topliceanu, A., Tramèr, F., & Zhang, F. (2021). *Chainlink 2.0: Next steps in the evolution of decentralized oracle networks* [White paper]. Chainlink Labs.

Buckley, D., Bendiab, G., Shiaeles, S., Savage, N., & Kolokotronis, N. (2021). CHAINGE: A blockchain solution to automate payment detail updates to subscription services. In *2021 IEEE International Conference on Communications Workshops (ICC Workshops)*. IEEE.

Buterin, V. (2014). *Ethereum: A next-generation smart contract and decentralized application platform* [White paper]. https://ethereum.org/whitepaper

Buterin, V., Weiss, Y., Tirosh, D., Nacson, S., Forshtat, A., Gazso, K., & Hess, T. (2021). *ERC-4337: Account abstraction using alt mempool* (Ethereum Improvement Proposal No. 4337). https://eips.ethereum.org/EIPS/eip-4337

Chang, W., Rocco, G., Millegan, B., Johnson, N., & Terbu, O. (2021). *EIP-4361: Sign-In with Ethereum* (Ethereum Improvement Proposal No. 4361). https://eips.ethereum.org/EIPS/eip-4361

Daian, P., Goldfeder, S., Kell, T., Li, Y., Zhao, X., Bentov, I., Breidenbach, L., & Juels, A. (2020). Flash Boys 2.0: Frontrunning in decentralized exchanges, miner extractable value, and consensus instability. In *2020 IEEE Symposium on Security and Privacy (SP)* (pp. 910–927). IEEE.

Gogol, K., Killer, C., Schlosser, M., Boeck, T., Stiller, B., & Tessone, C. (2024). *SoK: Decentralized finance (DeFi) – fundamentals, taxonomy and risks* (arXiv:2404.11281). arXiv.

Grybniak, S., Mihai, R., Datta, G., Goga, N., Mazurok, I., & Ozkul, O. F. (2022). Recurring payments on EVM-based platforms. In *2022 IEEE 1st Global Emerging Technology Blockchain Forum: Blockchain & Beyond (iGETblockchain)*. IEEE. https://doi.org/10.1109/iGETblockchain56591.2022.10087077

Kaleem, M., & Shi, W. (2021). Demystifying Pythia: A survey of ChainLink oracles usage on Ethereum. In *Financial Cryptography and Data Security. FC 2021 International Workshops* (Lecture Notes in Computer Science, Vol. 12676). Springer.

Lu, Q., Xu, X., Bandara, H. M. N. D., Chen, S., & Zhu, L. (2021). Patterns for blockchain-based payment applications. In *Proceedings of the 26th European Conference on Pattern Languages of Programs (EuroPLoP '21)*. ACM.

Lundfall, M. (2020). *ERC-2612: Permit extension for EIP-20 signed approvals* (Ethereum Improvement Proposal No. 2612). https://eips.ethereum.org/EIPS/eip-2612

Mayer, H., Bejarano, I., Fernandez, D., Ajzenman, G., Ayala, N., Santoalla, N., Sarraute, C., & Futoransky, A. (2020). *BatPay: A gas efficient protocol for the recurrent micropayment of ERC20 tokens* (arXiv:2002.02316). arXiv.

Meng, Z., & Feng, V. (2025). *A stream pipeline framework for digital payment programming based on smart contracts* (arXiv:2508.21075). arXiv.

Merkle, R. C. (1988). A digital signature based on a conventional encryption function. In *Advances in Cryptology – CRYPTO '87* (Lecture Notes in Computer Science, Vol. 293, pp. 369–378). Springer.

Mita, M., Ito, K., Ohsawa, S., & Tanaka, H. (2019). What is stablecoin?: A survey on price stabilization mechanisms for decentralized payment systems. In *2019 8th International Congress on Advanced Applied Informatics (IIAI-AAI)* (pp. 60–66). IEEE.

Nakamoto, S. (2008). *Bitcoin: A peer-to-peer electronic cash system* [White paper]. https://bitcoin.org/bitcoin.pdf

Onica, E., & Amariei, C. (2022). Using SGX for meta-transactions support in Ethereum DApps. In *Distributed Applications and Interoperable Systems (DAIS 2022)* (Lecture Notes in Computer Science). Springer.

OpenZeppelin. (2024). *OpenZeppelin Contracts* (Version 5.1.0) [Computer software]. https://github.com/OpenZeppelin/openzeppelin-contracts

Owocki, K. (2018). *ERC-948: Recurring subscription models are a better way of doing business* [GitHub issue #948]. ethereum/EIPs. https://github.com/ethereum/EIPs/issues/948

Pernice, I. G. A., Henningsen, S., Proskalovich, R., Florian, M., Elsden, H., & Scheuermann, B. (2019). Monetary stabilization in cryptocurrencies – design approaches and open questions. In *2019 Crypto Valley Conference on Blockchain Technology (CVCBT)* (pp. 47–59). IEEE.

Seres, I. A. (2020). *On blockchain metatransactions* (arXiv:2004.08094). arXiv.

Vogelsteller, F., & Buterin, V. (2015). *EIP-20: Token standard* (Ethereum Improvement Proposal No. 20). https://eips.ethereum.org/EIPS/eip-20

Wang, Q., & Chen, S. (2023). *Account abstraction, analysed* (arXiv:2309.00448). arXiv.

Wood, G. (2014). *Ethereum: A secure decentralised generalised transaction ledger* [Yellow paper]. https://ethereum.github.io/yellowpaper/paper.pdf

Appendices

Appendix A: SubscriptionVault.sol (complete source)

The complete source of the core contract as deployed and verified on Ethereum Sepolia at 0x9bC880415C54122cB64463888Db6a94Bf285dB8a.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {IERC20Permit} from "@openzeppelin/contracts/token/ERC20/extensions/IERC20Permit.sol";
import {SafeERC20} from "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
import {ReentrancyGuard} from "@openzeppelin/contracts/utils/ReentrancyGuard.sol";
import {PlanRegistry} from "./PlanRegistry.sol";

/// @title SubscriptionVault
/// @notice Pull-based recurring payments. A subscriber enrols once (optionally with an
///         EIP-2612 permit). Afterwards ANY account (a "keeper") may trigger a charge once
///         it is due. The contract, not the allowance, enforces the limit: at most
///         `plan.price`, at most once per elapsed period, on an anchored schedule.
contract SubscriptionVault is ReentrancyGuard {
    using SafeERC20 for IERC20;

    enum Status {
        None,
        Active,
        PastDue,
        Lapsed,
        Cancelled
    }

    struct Subscription {
        address subscriber;
        uint64 planId;
        uint64 startedAt;
        uint64 nextChargeAt; // anchored schedule: advances by exactly `period` per successful charge
        uint64 pastDueSince; // 0 unless PastDue
        uint32 chargeCount; // includes the initial payment taken at subscribe time
        Status status;
    }

    /// @dev Immutable plan terms cached on first use (they can never change in the registry).
    struct Terms {
        address payee;
        uint96 price;
        address token;
        uint32 period;
        uint32 gracePeriod;
    }

    enum Outcome {
        Charged,
        Failed,
        Lapsed
    }

    uint256 public constant MAX_BATCH = 200;
    uint16 public constant MAX_KEEPER_FEE_BPS = 1_000; // 10%

    PlanRegistry public immutable registry;
    /// @notice Keeper reward in basis points of `price`, paid to the caller of a successful charge.
    uint16 public immutable keeperFeeBps;

    uint256 private _subCount;
    mapping(uint256 subId => Subscription) private _subs;
    mapping(address subscriber => mapping(uint256 planId => uint256 subId)) private _latestSub;
    mapping(address account => mapping(address token => uint256)) private _claimable;
    mapping(uint256 planId => Terms) private _terms;

    event Subscribed(
        uint256 indexed subId, uint256 indexed planId, address indexed subscriber, uint256 amount, uint64 nextChargeAt
    );
    event Charged(
        uint256 indexed subId,
        uint256 indexed planId,
        uint256 amount,
        uint256 fee,
        address indexed keeper,
        uint64 newNextChargeAt
    );
```

```
75      event ChargeFailed(uint256 indexed subId, uint8 reason); // 1 = balance, 2 = allowance
76      event SubscriptionLapsed(uint256 indexed subId);
77      event Cancelled(uint256 indexed subId, uint64 paidThrough);
78      event Withdrawn(address indexed account, address indexed token, uint256 amount);
79
80      error NotDue();
81      error NotChargeable(Status status);
82      error NotSubscriber();
83      error NothingToWithdraw();
84      error PlanInactive(uint256 planId);
85      error AlreadySubscribed(uint256 subId);
86      error BatchTooLarge(uint256 size);
87      error FeeTooHigh();
88
89      uint8 internal constant REASON_BALANCE = 1;
90      uint8 internal constant REASON_ALLOWANCE = 2;
91
92      constructor(PlanRegistry registry_, uint16 keeperFeeBps_) {
93          if (keeperFeeBps_ > MAX_KEEPER_FEE_BPS) revert FeeTooHigh();
94          registry = registry_;
95          keeperFeeBps = keeperFeeBps_;
96      }
97
98      // ------------------------------------------------------------------
99      // Subscribe
100     // ------------------------------------------------------------------
101
102     /// @notice Enrol with one EIP-2612 signature. If the permit was front-run (already
used),
103     ///         enrolment still succeeds as long as the resulting allowance is sufficient.
104     function subscribeWithPermit(uint256 planId, uint256 permitValue, uint256 deadline, uint8
v, bytes32 r, bytes32 s)
105         external
106         nonReentrant
107         returns (uint256 subId)
108     {
109         Terms memory t = _loadTerms(planId);
110         try IERC20Permit(t.token).permit(msg.sender, address(this), permitValue, deadline, v,
r, s) {} catch {}
111         return _subscribe(planId, t);
112     }
113
114     /// @notice Enrol using an allowance the subscriber already granted.
115     function subscribe(uint256 planId) external nonReentrant returns (uint256 subId) {
116         return _subscribe(planId, _loadTerms(planId));
117     }
118
119     function _subscribe(uint256 planId, Terms memory t) private returns (uint256 subId) {
120         (,,,,, bool active) = registry.getPlanTerms(planId);
121         if (!active) revert PlanInactive(planId);
122
123         uint256 existing = _latestSub[msg.sender][planId];
124         if (existing != 0) {
125             Status st = _subs[existing].status;
126             if (st == Status.Active || st == Status.PastDue) revert
AlreadySubscribed(existing);
127         }
128
129         subId = ++_subCount;
130         uint64 nowTs = uint64(block.timestamp);
131         uint64 next = nowTs + t.period;
132         _subs[subId] = Subscription({
133             subscriber: msg.sender,
134             planId: uint64(planId),
135             startedAt: nowTs,
136             nextChargeAt: next,
137             pastDueSince: 0,
138             chargeCount: 1,
139             status: Status.Active
140         });
141         _latestSub[msg.sender][planId] = subId;
142
143         // First period is paid immediately by the subscriber; no keeper fee.
144         IERC20(t.token).safeTransferFrom(msg.sender, address(this), t.price);
145         _claimable[t.payee][t.token] += t.price;
146
147         emit Subscribed(subId, planId, msg.sender, t.price, next);
148     }
149
150     // ------------------------------------------------------------------
151     // Charge
152     // ------------------------------------------------------------------
153
154     /// @notice Collect one period from `subId`. Callable by anyone once due.
155     function charge(uint256 subId) external nonReentrant {
156         Subscription storage s = _subs[subId];
157         Status st = s.status;
158         if (st != Status.Active && st != Status.PastDue) revert NotChargeable(st);
159         if (block.timestamp < s.nextChargeAt) revert NotDue();
160         _charge(subId, s);
161     }
162
163     /// @notice Charge many subscriptions. Never reverts on a single bad id.
164     /// @return ok successful charges
165     /// @return failed transitions to PastDue or Lapsed
```

```
    /// @return skipped ids that were not due or not chargeable
    function chargeBatch(uint256[] calldata subIds)
        external
        nonReentrant
        returns (uint256 ok, uint256 failed, uint256 skipped)
    {
        uint256 n = subIds.length;
        if (n > MAX_BATCH) revert BatchTooLarge(n);
        for (uint256 i; i < n; ++i) {
            Subscription storage s = _subs[subIds[i]];
            Status st = s.status;
            if ((st != Status.Active && st != Status.PastDue) || block.timestamp <
s.nextChargeAt) {
                ++skipped;
                continue;
            }
            if (_charge(subIds[i], s) == Outcome.Charged) ++ok;
            else ++failed;
        }
    }

    /// @dev Caller has verified the subscription is chargeable and due.
    function _charge(uint256 subId, Subscription storage s) private returns (Outcome) {
        Terms memory t = _terms[s.planId];
        address subscriber = s.subscriber;
        IERC20 token = IERC20(t.token);

        uint8 reason;
        if (token.balanceOf(subscriber) < t.price) reason = REASON_BALANCE;
        else if (token.allowance(subscriber, address(this)) < t.price) reason =
REASON_ALLOWANCE;

        if (reason != 0) {
            if (block.timestamp >= uint256(s.nextChargeAt) + t.gracePeriod) {
                s.status = Status.Lapsed;
                s.pastDueSince = 0;
                emit SubscriptionLapsed(subId);
                return Outcome.Lapsed;
            }
            if (s.status != Status.PastDue) {
                s.status = Status.PastDue;
                s.pastDueSince = s.nextChargeAt;
            }
            emit ChargeFailed(subId, reason);
            return Outcome.Failed;
        }

        uint64 next = s.nextChargeAt + t.period;
        s.nextChargeAt = next;
        s.chargeCount += 1;
        s.status = Status.Active;
        s.pastDueSince = 0;

        uint256 fee = (uint256(t.price) * keeperFeeBps) / 10_000;
        token.safeTransferFrom(subscriber, address(this), t.price);
        _claimable[t.payee][t.token] += t.price - fee;
        if (fee != 0) _claimable[msg.sender][t.token] += fee;

        emit Charged(subId, s.planId, t.price, fee, msg.sender, next);
        return Outcome.Charged;
    }

    // ------------------------------------------------------------------
    // Cancel / withdraw
    // ------------------------------------------------------------------

    /// @notice Stop future charges. Access continues until the paid-through time
(`nextChargeAt`).
    function cancel(uint256 subId) external {
        Subscription storage s = _subs[subId];
        if (msg.sender != s.subscriber) revert NotSubscriber();
        Status st = s.status;
        if (st != Status.Active && st != Status.PastDue) revert NotChargeable(st);
        s.status = Status.Cancelled;
        s.pastDueSince = 0;
        emit Cancelled(subId, s.nextChargeAt);
    }

    /// @notice Pay out msg.sender's own claimable balance of `token`.
    function withdraw(address token) external nonReentrant {
        uint256 amount = _claimable[msg.sender][token];
        if (amount == 0) revert NothingToWithdraw();
        _claimable[msg.sender][token] = 0;
        IERC20(token).safeTransfer(msg.sender, amount);
        emit Withdrawn(msg.sender, token, amount);
    }

    // ------------------------------------------------------------------
    // Views
    // ------------------------------------------------------------------

    function isEntitled(uint256 subId) public view returns (bool) {
        Subscription storage s = _subs[subId];
        Status st = s.status;
        if (st == Status.Active || st == Status.PastDue) {
```

```
258              return block.timestamp < uint256(s.nextChargeAt) + _terms[s.planId].gracePeriod;
259          }
260          if (st == Status.Cancelled) return block.timestamp < s.nextChargeAt;
261          return false;
262      }
263
264      /// @notice The subscriber's entitled subscription to `planId`, or 0 if none.
265      function entitledSubOf(address subscriber, uint256 planId) external view returns (uint256
subId) {
266          subId = _latestSub[subscriber][planId];
267          if (subId != 0 && !isEntitled(subId)) subId = 0;
268      }
269
270      /// @notice The subscriber's most recent subscription to `planId` (any status), or 0.
271      function latestSubOf(address subscriber, uint256 planId) external view returns (uint256)
{
272          return _latestSub[subscriber][planId];
273      }
274
275      function getSubscription(uint256 subId) external view returns (Subscription memory) {
276          return _subs[subId];
277      }
278
279      function claimable(address account, address token) external view returns (uint256) {
280          return _claimable[account][token];
281      }
282
283      /// @notice Scan up to `limit` subscription ids starting at `cursor` and return those
that are
284      ///         chargeable and due. `nextCursor` is 0 once the scan reached the end.
285      function dueSubscriptions(uint256 cursor, uint256 limit)
286          external
287          view
288          returns (uint256[] memory ids, uint256 nextCursor)
289      {
290          uint256 total = _subCount;
291          uint256 start = cursor == 0 ? 1 : cursor;
292          if (start > total || limit == 0) return (new uint256[](0), 0);
293          uint256 end = start + limit - 1;
294          if (end > total) end = total;
295
296          uint256[] memory buf = new uint256[](end - start + 1);
297          uint256 count;
298          for (uint256 id = start; id <= end; ++id) {
299              Subscription storage s = _subs[id];
300              Status st = s.status;
301              if ((st == Status.Active || st == Status.PastDue) && block.timestamp >=
s.nextChargeAt) {
302                  buf[count++] = id;
303              }
304          }
305          assembly {
306              mstore(buf, count)
307          }
308          return (buf, end < total ? end + 1 : 0);
309      }
310
311      function subscriptionCount() external view returns (uint256) {
312          return _subCount;
313      }
314
315      // ---------------------------------------------------------------------
316      // Internal
317      // ---------------------------------------------------------------------
318
319      function _loadTerms(uint256 planId) private returns (Terms memory t) {
320          t = _terms[planId];
321          if (t.token == address(0)) {
322              (address payee, address token, uint96 price, uint32 period, uint32 grace,) =
registry.getPlanTerms(planId);
323              t = Terms({payee: payee, price: price, token: token, period: period, gracePeriod:
grace});
324              _terms[planId] = t;
325          }
326      }
327  }
328
```

Appendix B: PlanRegistry.sol and UsageRootRegistry.sol

B.1 PlanRegistry.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

/// @title PlanRegistry
/// @notice Providers publish subscription plans. Economic terms (token, price, period,
///         grace, payee) are immutable once created; there is deliberately no setter.
contract PlanRegistry {
    struct Plan {
        address provider; // msg.sender at creation
        address payee; // receives revenue; immutable
        address token; // ERC-20 settlement token; immutable
        uint96 price; // per-period price in token units; immutable
        uint32 period; // seconds; immutable; > 0
        uint32 gracePeriod; // seconds; immutable; MUST be < period
        bool active; // provider can close to NEW subscriptions only
        string metadataURI; // name/description/features (JSON or ipfs://)
    }

    /// @dev Plan ids start at 1 so that 0 can mean "no plan".
    Plan[] private _plans;

    event PlanCreated(
        uint256 indexed planId,
        address indexed provider,
        address indexed payee,
        address token,
        uint96 price,
        uint32 period,
        uint32 gracePeriod,
        string metadataURI
    );
    event PlanActiveSet(uint256 indexed planId, bool active);

    error ZeroAddress();
    error ZeroPrice();
    error ZeroPeriod();
    error GraceNotLessThanPeriod(uint32 gracePeriod, uint32 period);
    error UnknownPlan(uint256 planId);
    error NotProvider();

    function createPlan(
        address token,
        uint96 price,
        uint32 period,
        uint32 gracePeriod,
        address payee,
        string calldata metadataURI
    ) external returns (uint256 planId) {
        if (token == address(0) || payee == address(0)) revert ZeroAddress();
        if (price == 0) revert ZeroPrice();
        if (period == 0) revert ZeroPeriod();
        if (gracePeriod >= period) revert GraceNotLessThanPeriod(gracePeriod, period);

        _plans.push(
            Plan({
                provider: msg.sender,
                payee: payee,
                token: token,
                price: price,
                period: period,
                gracePeriod: gracePeriod,
                active: true,
                metadataURI: metadataURI
            })
        );
        planId = _plans.length;
        emit PlanCreated(planId, msg.sender, payee, token, price, period, gracePeriod,
metadataURI);
    }

    /// @notice Open or close a plan to NEW subscriptions. Existing subscriptions are
unaffected.
    function setPlanActive(uint256 planId, bool active) external {
        Plan storage p = _plan(planId);
        if (msg.sender != p.provider) revert NotProvider();
        p.active = active;
        emit PlanActiveSet(planId, active);
    }

    function getPlan(uint256 planId) external view returns (Plan memory) {
        return _plan(planId);
    }

    /// @notice Economic terms without the metadata string (cheaper for on-chain callers).
    function getPlanTerms(uint256 planId)
        external
        view
```

```
 86          returns (address payee, address token, uint96 price, uint32 period, uint32
gracePeriod, bool active)
 87      {
 88          Plan storage p = _plan(planId);
 89          return (p.payee, p.token, p.price, p.period, p.gracePeriod, p.active);
 90      }
 91
 92      function planCount() external view returns (uint256) {
 93          return _plans.length;
 94      }
 95
 96      function _plan(uint256 planId) private view returns (Plan storage) {
 97          if (planId == 0 || planId > _plans.length) revert UnknownPlan(planId);
 98          return _plans[planId - 1];
 99      }
100  }
101
```

B.2 UsageRootRegistry.sol

```
  1  // SPDX-License-Identifier: MIT
  2  pragma solidity ^0.8.24;
  3
  4  import {Ownable} from "@openzeppelin/contracts/access/Ownable.sol";
  5  import {MerkleProof} from "@openzeppelin/contracts/utils/cryptography/MerkleProof.sol";
  6
  7  /// @title UsageRootRegistry
  8  /// @notice Gateways commit one Merkle root of metered usage rows per (planId, epoch).
  9  ///         Leaves follow the OpenZeppelin merkle-tree JS library StandardMerkleTree (double-
hashed) with type
 10  ///         ["uint64 epoch","address wallet","uint256 subId","string model","uint32
promptTokens",
 11  ///          "uint32 completionTokens","bytes32 requestHash","bytes32 responseHash","uint64
timestamp"].
 12  contract UsageRootRegistry is Ownable {
 13      struct UsageRow {
 14          uint64 epoch;
 15          address wallet;
 16          uint256 subId;
 17          string model;
 18          uint32 promptTokens;
 19          uint32 completionTokens;
 20          bytes32 requestHash;
 21          bytes32 responseHash;
 22          uint64 timestamp;
 23      }
 24
 25      struct EpochRecord {
 26          bytes32 root;
 27          uint64 rowCount;
 28          uint128 totalTokens;
 29          uint64 committedAt;
 30          address committer;
 31          string uri;
 32      }
 33
 34      mapping(address => bool) public isCommitter;
 35      mapping(uint256 planId => mapping(uint64 epoch => EpochRecord)) private _epochs;
 36
 37      event CommitterSet(address indexed committer, bool allowed);
 38      event RootCommitted(
 39          uint256 indexed planId, uint64 indexed epoch, bytes32 root, uint64 rowCount, uint128
totalTokens, string uri
 40      );
 41
 42      error NotCommitter();
 43      error ZeroRoot();
 44      error RootAlreadyCommitted(uint256 planId, uint64 epoch);
 45
 46      constructor(address owner_) Ownable(owner_) {}
 47
 48      function setCommitter(address committer, bool allowed) external onlyOwner {
 49          isCommitter[committer] = allowed;
 50          emit CommitterSet(committer, allowed);
 51      }
 52
 53      function commitRoot(
 54          uint256 planId,
 55          uint64 epoch,
 56          bytes32 root,
 57          uint64 rowCount,
 58          uint128 totalTokens,
 59          string calldata uri
 60      ) external {
 61          if (!isCommitter[msg.sender]) revert NotCommitter();
 62          if (root == bytes32(0)) revert ZeroRoot();
 63          EpochRecord storage e = _epochs[planId][epoch];
 64          if (e.root != bytes32(0)) revert RootAlreadyCommitted(planId, epoch);
 65          e.root = root;
 66          e.rowCount = rowCount;
 67          e.totalTokens = totalTokens;
```

```
 68          e.committedAt = uint64(block.timestamp);
 69          e.committer = msg.sender;
 70          e.uri = uri;
 71          emit RootCommitted(planId, epoch, root, rowCount, totalTokens, uri);
 72      }
 73
 74      /// @notice Verify a (double-hashed) StandardMerkleTree leaf against the committed root.
 75      function verifyRow(uint256 planId, uint64 epoch, bytes32[] calldata proof, bytes32 leaf)
 76          external
 77          view
 78          returns (bool)
 79      {
 80          bytes32 root = _epochs[planId][epoch].root;
 81          if (root == bytes32(0)) return false;
 82          return MerkleProof.verifyCalldata(proof, root, leaf);
 83      }
 84
 85      function rootOf(uint256 planId, uint64 epoch) external view returns (bytes32) {
 86          return _epochs[planId][epoch].root;
 87      }
 88
 89      function getEpoch(uint256 planId, uint64 epoch) external view returns (EpochRecord
memory) {
 90          return _epochs[planId][epoch];
 91      }
 92
 93      /// @notice StandardMerkleTree leaf hash of a usage row, so anyone can recompute it on-
chain.
 94      function hashRow(UsageRow calldata r) external pure returns (bytes32) {
 95          return keccak256(
 96              bytes.concat(
 97                  keccak256(
 98                      abi.encode(
 99                          r.epoch,
100                          r.wallet,
101                          r.subId,
102                          r.model,
103                          r.promptTokens,
104                          r.completionTokens,
105                          r.requestHash,
106                          r.responseHash,
107                          r.timestamp
108                      )
109                  )
110              )
111          );
112      }
113  }
114
```

Appendix C: Keeper core (keeper/src/keeper.ts)

```
import { appendFileSync, mkdirSync } from "node:fs";
import { dirname } from "node:path";
import {
  parseEventLogs,
  type Account,
  type Address,
  type Chain,
  type Hash,
  type PublicClient,
  type Transport,
  type WalletClient,
} from "viem";
import { subscriptionVaultAbi } from "@cashew/shared";

export type Outcome = "charged" | "failed" | "lapsed" | "skipped";

export interface KeeperLogLine {
  ts: number; // wall clock ms
  subId: number;
  scheduledAt: number; // the nextChargeAt that was due
  executedAt: number; // block timestamp of inclusion
  driftSeconds: number;
  outcome: Outcome;
  reason?: string;
  txHash: Hash;
  gasUsed: number; // gas of the whole chargeBatch tx
  batchSize: number;
}

export interface TickResult {
  startedAt: number;
  due: number;
  charged: number;
  failed: number;
  lapsed: number;
  skipped: number;
  txs: number;
  gasUsed: bigint;
  lines: KeeperLogLine[];
}

export interface KeeperOptions {
  publicClient: PublicClient<Transport, Chain>;
  walletClient: WalletClient<Transport, Chain, Account>;
  vault: Address;
  batchSize?: number;
  pageSize?: number;
  /** Max re-scan rounds per tick (lets a late keeper catch up several periods in one tick).
*/
  maxRounds?: number;
  logFile?: string | null;
  log?: (msg: string) => void;
}

/**
 * Permissionless keeper: finds due subscriptions and charges them in batches.
 * It holds no privileges — the vault enforces amount and timing.
 */
export class Keeper {
  readonly opts: Required<Omit<KeeperOptions, "logFile" | "log">> & Pick<KeeperOptions,
"logFile" | "log">;
  lastTick: TickResult | null = null;
  lastQueueSize = 0;
  ticking = false;

  constructor(opts: KeeperOptions) {
    this.opts = { batchSize: 100, pageSize: 500, maxRounds: 16, ...opts };
    if (opts.logFile) mkdirSync(dirname(opts.logFile), { recursive: true });
  }

  /** Page through dueSubscriptions(cursor, pageSize). */
  async findDue(): Promise<bigint[]> {
    const { publicClient, vault, pageSize } = this.opts;
    const ids: bigint[] = [];
    let cursor = 0n;
    do {
      const [page, next] = await publicClient.readContract({
        address: vault,
        abi: subscriptionVaultAbi,
        functionName: "dueSubscriptions",
        args: [cursor, BigInt(pageSize)],
      });
      ids.push(...page);
      cursor = next;
    } while (cursor !== 0n);
    return ids;
  }

  async tick(): Promise<TickResult> {
    if (this.ticking) throw new Error("tick already in progress");
    this.ticking = true;
```

```
90      const res: TickResult = {
91        startedAt: Date.now(),
92        due: 0,
93        charged: 0,
94        failed: 0,
95        lapsed: 0,
96        skipped: 0,
97        txs: 0,
98        gasUsed: 0n,
99        lines: [],
100     };
101     try {
102       // Subs that failed this tick (PastDue) stay "due"; don't retry them until the next
tick.
103       const triedFailed = new Set<bigint>();
104       for (let round = 0; round < this.opts.maxRounds; round++) {
105         const due = (await this.findDue()).filter((id) => !triedFailed.has(id));
106         if (round === 0) this.lastQueueSize = due.length;
107         if (due.length === 0) break;
108         res.due += due.length;
109         for (let i = 0; i < due.length; i += this.opts.batchSize) {
110           const chunk = due.slice(i, i + this.opts.batchSize);
111           const lines = await this.chargeChunk(chunk);
112           res.txs++;
113           res.gasUsed += BigInt(lines[0]?.gasUsed ?? 0);
114           for (const l of lines) {
115             res.lines.push(l);
116             if (l.outcome === "charged") res.charged++;
117             else if (l.outcome === "failed") (res.failed++,
triedFailed.add(BigInt(l.subId)));
118             else if (l.outcome === "lapsed") res.lapsed++;
119             else (res.skipped++, triedFailed.add(BigInt(l.subId)));
120           }
121         }
122       }
123     } finally {
124       this.ticking = false;
125     }
126     this.lastTick = res;
127     return res;
128   }
129
130   private async chargeChunk(ids: bigint[]): Promise<KeeperLogLine[]> {
131     const { publicClient, walletClient, vault } = this.opts;
132     const subs = await Promise.all(
133       ids.map((id) =>
134         publicClient.readContract({ address: vault, abi: subscriptionVaultAbi, functionName:
"getSubscription", args: [id] }),
135       ),
136     );
137     const scheduled = new Map(ids.map((id, i) => [id, Number(subs[i].nextChargeAt)]));
138
139     const { request } = await publicClient.simulateContract({
140       account: walletClient.account,
141       address: vault,
142       abi: subscriptionVaultAbi,
143       functionName: "chargeBatch",
144       args: [ids],
145     });
146     const hash = await walletClient.writeContract(request);
147     const receipt = await publicClient.waitForTransactionReceipt({ hash });
148     if (receipt.status !== "success") throw new Error(`chargeBatch reverted: ${hash}`);
149     const block = await publicClient.getBlock({ blockNumber: receipt.blockNumber });
150     const executedAt = Number(block.timestamp);
151
152     const events = parseEventLogs({ abi: subscriptionVaultAbi, logs: receipt.logs });
153     const outcome = new Map<bigint, { outcome: Outcome; reason?: string }>();
154     for (const ev of events) {
155       if (ev.eventName === "Charged") outcome.set(ev.args.subId, { outcome: "charged" });
156       else if (ev.eventName === "ChargeFailed")
157         outcome.set(ev.args.subId, { outcome: "failed", reason: ev.args.reason === 1 ?
"balance" : "allowance" });
158       else if (ev.eventName === "SubscriptionLapsed") outcome.set(ev.args.subId, { outcome:
"lapsed" });
159     }
160
161     const lines: KeeperLogLine[] = ids.map((id) => {
162       const o = outcome.get(id) ?? { outcome: "skipped" as const };
163       const scheduledAt = scheduled.get(id)!;
164       return {
165         ts: Date.now(),
166         subId: Number(id),
167         scheduledAt,
168         executedAt,
169         driftSeconds: executedAt - scheduledAt,
170         ...o,
171         txHash: hash,
172         gasUsed: Number(receipt.gasUsed),
173         batchSize: ids.length,
174       };
175     });
176     if (this.opts.logFile) appendFileSync(this.opts.logFile, lines.map((l) =>
JSON.stringify(l)).join("\n") + "\n");
177     const c = lines.filter((l) => l.outcome === "charged").length;
```

```
      this.opts.log?.(`chargeBatch(${ids.length}) → ${c} charged, ${lines.length - c} other · gas ${receipt.gasUsed} · ${hash}`);
      return lines;
    }
  }

```

Appendix D: Invariant specifications (excerpt)

The five protocol invariants as encoded for Foundry's handler-based invariant fuzzer. The handler that drives random action sequences is in contracts/test/invariant/Handler.sol.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import {Test, console} from "forge-std/Test.sol";
import {StdInvariant} from "forge-std/StdInvariant.sol";
import {MockUSDC} from "../../src/MockUSDC.sol";
import {PlanRegistry} from "../../src/PlanRegistry.sol";
import {SubscriptionVault} from "../../src/SubscriptionVault.sol";
import {Handler} from "./Handler.sol";

/// forge-config: default.invariant.runs = 256
/// forge-config: default.invariant.depth = 64
contract InvariantsTest is StdInvariant, Test {
    MockUSDC usdc;
    PlanRegistry registry;
    SubscriptionVault vault;
    Handler handler;

    function setUp() public {
        vm.warp(1_700_000_000);
        usdc = new MockUSDC();
        registry = new PlanRegistry();
        vault = new SubscriptionVault(registry, 25);
        handler = new Handler(usdc, registry, vault);
        targetContract(address(handler));
    }

    /// I1: for every subscriber, total pulled <= Σ_subs price × (1 + floor((now - startedAt)/period)).
    function invariant_I1_noOvercharge() public view {
        uint256 n = vault.subscriptionCount();
        for (uint256 i; i < handler.subscriberCount(); ++i) {
            address who = handler.subscribers(i);
            uint256 bound_;
            for (uint256 id = 1; id <= n; ++id) {
                SubscriptionVault.Subscription memory s = vault.getSubscription(id);
                if (s.subscriber != who) continue;
                (,, uint96 price, uint32 period,,) = registry.getPlanTerms(s.planId);
                bound_ += uint256(price) * (1 + (block.timestamp - s.startedAt) / period);
            }
            assertLe(handler.ghostPulled(who), bound_, "I1 overcharge");
        }
    }

    /// I2: no successful charge ever happens after a subscription is Cancelled or Lapsed.
    function invariant_I2_noChargeAfterTerminal() public view {
        assertFalse(handler.ghostChargedAfterTerminal(), "I2 charged after terminal");
        uint256 n = vault.subscriptionCount();
        for (uint256 id = 1; id <= n; ++id) {
            if (!handler.ghostTerminal(id)) continue;
            SubscriptionVault.Subscription memory s = vault.getSubscription(id);
            assertEq(s.chargeCount, handler.ghostTerminalCount(id), "I2 chargeCount moved");
            assertTrue(
                s.status == SubscriptionVault.Status.Cancelled || s.status == SubscriptionVault.Status.Lapsed,
                "I2 left terminal state"
            );
        }
    }

    /// I3: vault token balance == Σ claimable.
    function invariant_I3_solvency() public view {
        address[] memory accts = handler.allAccounts();
        uint256 sum;
        for (uint256 i; i < accts.length; ++i) sum += vault.claimable(accts[i], address(usdc));
        assertEq(usdc.balanceOf(address(vault)), sum, "I3 insolvent");
    }

    /// I4: nextChargeAt == startedAt + period × chargeCount for every subscription.
    function invariant_I4_anchoredSchedule() public view {
        uint256 n = vault.subscriptionCount();
        for (uint256 id = 1; id <= n; ++id) {
            SubscriptionVault.Subscription memory s = vault.getSubscription(id);
            (,,, uint32 period,,) = registry.getPlanTerms(s.planId);
            assertEq(s.nextChargeAt, uint256(s.startedAt) + uint256(period) * s.chargeCount, "I4 schedule drift");
        }
    }

    /// I5: no account ever withdraws more than it was credited; the attacker is never paid.
    function invariant_I5_ownFundsOnly() public view {
```

```
 79          address[] memory accts = handler.allAccounts();
 80          for (uint256 i; i < accts.length; ++i) {
 81              address a = accts[i];
 82              assertLe(handler.ghostWithdrawn(a), handler.ghostCredited(a), "I5 withdrew more
than credited");
 83              assertEq(
 84                  handler.ghostCredited(a) - handler.ghostWithdrawn(a),
 85                  vault.claimable(a, address(usdc)),
 86                  "I5 claimable mismatch"
 87              );
 88          }
 89          assertFalse(handler.ghostAttackerWithdrew(), "I5 attacker withdrew");
 90      }
 91
 92      function afterInvariant() external view {
 93          uint256 n = vault.subscriptionCount();
 94          uint256 charges;
 95          uint256[5] memory byStatus;
 96          for (uint256 id = 1; id <= n; ++id) {
 97              SubscriptionVault.Subscription memory s = vault.getSubscription(id);
 98              charges += s.chargeCount - 1;
 99              byStatus[uint8(s.status)]++;
100          }
101          console.log("subscriptions", n, "keeper charges", charges);
102          console.log("active/pastDue", byStatus[1], byStatus[2]);
103          console.log("lapsed/cancelled", byStatus[3], byStatus[4]);
104          console.log("charge calls", handler.calls("charge"), "batch calls",
handler.calls("chargeBatch"));
105      }
106  }
107
```

Appendix E: Deployment record and reproduction commands

Table E.1 Ethereum Sepolia deployment (chainId 11155111)

Item	Value
Deployer / provider / payee	0xdb0C7Acf5560eA87eD7D5D7C570dF21092ac62E7
MockUSDC	0x75e8B2f30a7c08cf3967A2745dFAb0CF418Ced96
PlanRegistry	0x9109E3Af83D6499E4d433128B60CDAc5505Cc815
SubscriptionVault	0x9bC880415C54122cB64463888Db6a94Bf285dB8a
UsageRootRegistry	0x3fF143e67616114B4F638b332BaF060Cb8e8696C
Gateway (root committer)	0x6c9e15ce9e9B812d7746DDab4926b2eBFE9fade2
Keeper	0x9EE946ea5d8aDf194295a1a951bCd2F9C7CD2159
First deployment block	11925719
Plans	#1 demo: 20 mUSDC / 180 s / grace 60 s; #2 production: 20 mUSDC / 30 days / grace 3 days

Reproducing the results. From a fresh clone of the repository:

```
pnpm i                        # install the workspace
pnpm test:contracts           # 72 Foundry tests (unit, fuzz, invariant)
pnpm local:up                 # anvil + deploy + seed + keeper + gateway (mock) +
dashboard
pnpm test:gateway             # 10 end-to-end gateway tests against the local chain
pnpm results:local            # test summary, gas, batch sweep, cost, schedule sim,
outage
pnpm deploy:testnet           # deploy + seed + verify on Ethereum Sepolia
pnpm testnet:up               # keeper + gateway + dashboard against Sepolia
pnpm results:testnet          # lifecycle, grid snapshot, live bench (asks first),
epochs
pnpm results:summary          # figures + results/RESULTS.md
```